Basi di dati — Tecnologie

Università di Verona
Imbriani Paolo - VR500437
Professor Sara Migliorini

April 14, 2026

Contents

1 SQL **4**

1.1 Istruzioni 4
1.2 SELECT 5
1.3 FROM 5
1.4 WHERE 6
1.4.1 LIKE 6
1.4.2 BETWEEN 6
1.4.3 SIMILAR TO 6
1.4.4 IN 7
1.4.5 IS NULL 7
1.5 SQL e Algebra Relazionale 7
1.5.1 Duplicati 7
1.5.2 Selezione, Proiezione, Join 8
1.6 JOIN 8
1.6.1 INNER JOIN 8
1.6.2 LEFT [OUTER] JOIN 9
1.6.3 RIGHT [OUTER] JOIN 9
1.6.4 FULL [OUTER] JOIN 9
1.7 ALIAS 9
1.8 ORDER BY 9
1.9 OPERAZIONI INSIEMISTICHE 10
1.10 OPERATORI AGGREGATI 10
1.10.1 COUNT 10
1.10.2 MAX, MIN, AVG, SUM 10
1.11 OPERATORI DI RAGGRUPPAMENTO 11
1.11.1 GROUP BY 11
1.11.2 HAVING 11
1.12 Interrogazioni nidificate 12
1.12.1 Nidificazione nella clausola WHERE 12
1.12.2 Nidificazione nidifiicate Binding e EXISTS 12
1.13 VIEW 14
1.14 Esercizi 14

2 Metodi di accesso **18**

2.1 Operazioni sulle pagine 18
2.2 Criteri di organizzazione 19
2.2.1 Struttura primaria sequenziale/seriale 19

2.3 Indici 20
2.4 Minimizzare i costi di accesso 22
2.4.1 B+TREE 22
2.4.2 Ricerca in un B+TREE 25
2.4.3 Inserimento in un B+TREE 26
2.4.4 Cancellazione 27
2.4.5 Hash 30
2.4.6 Ricerca in un indice hash 31
2.4.7 Collisioni 32

3 Transazioni concorrenti **33**
3.1 Anomalie 34
3.1.1 Perdita di aggiornamento 34
3.1.2 Lettura inconsistente 35
3.1.3 Lettura sporca 36
3.1.4 Aggiornamento fantasma 37
3.1.5 Inserimento fantasma 38
3.2 Schedule 38
3.2.1 Schedule seriale 38
3.3 Tecniche di gestione della concorrenza 39
3.3.1 View-equivalenza 39
3.3.2 Conflict-equivalenza 46
3.3.3 Locking a due fasi stretto 52
3.3.4 Blocco critico (deadlock) 54
3.3.5 Gestione della concorrenza in SQL 56

1 SQL

SQL è il linguaggio di riferimento per le basi di dati relazionali. Originariamente SQL era l'acronimo di "Structured Query Language", ma oggi è semplicemente SQL. SQL ha varie funzionalità:

- **Data Definition Language (DDL)**: consente di definire la struttura del database, come la creazione di tabelle, l'aggiunta di colonne e la definizione di chiavi primarie.
- **Data Manipulation Language (DML)**: consente di manipolare i dati all'interno del database, come l'inserimento, l'aggiornamento e la cancellazione di record.
- **Data Query Language (DQL)**: consente di interrogare il database per recuperare informazioni specifiche.

Ne esistono inoltre varie versioni. In laboratorio useremo SQL-2 poiché sappiamo che è supportato da tutti i DBMS.

1.1 Istruzioni

La sintassi di base è:

```
SELECT [ DISTINCT ]
[ * | expression [[ AS] output_name ] [, ...] ]
[ FROM from_item [, ...] ]
[ WHERE condition ]
[ GROUP BY grouping_element [, ...] ]
[ HAVING condition [, ...] ]
[ { UNION | INTERSECT | EXCEPT } [ DISTINCT ] other_select ]
[ ORDER BY expression [ ASC | DESC | USING operator ]]
```

- expression è un' espressione che determina un attributo.
- from_item è un' espressione che determina una sorgente per gli attributi.
- condition è un' espressione booleana per selezionare i valori degli attributi.
- grouping_element è un'espressione per poter eseguire operazioni su più valori di un attributo e considerare il risultato
- SELECT, FROM e WHERE sono chiamate clausole.
- Estrae dal prodotto cartesiano delle tabelle elencate nella clausola FROM, le righe che soddisfano la condizione specificata nella clausola WHERE.

– Il risultato dell'interrogazione è una tabella con una riga per ogni riga prodotta dalla clausola FROM ed eventualmente filtrata dalla clausola WHERE (se esiste), le cui colonne dipendono dalla lista degli attributi nella clausola SELECT.

1.2 SELECT

```
SELECT [ DISTINCT ]
[{* | expression [[ AS] output_name ]} [, ...]]
```

- è un'abbreviazione per indicare tutti gli attributi delle tabelle.
- expression è un'espressione che coinvolge gli attributi della tabella (può anche essere semplicemente il nome di un attributo).
- output_name è il nome assegnato all'attributo che conterrà il risultato della valutazione dell'espressione expression nella relazione risultato.
- DISTINCT: se presente richiede l'eliminazione delle tuple duplicate.

1.3 FROM

```
FROM from_item [, ...]
```

1. `table_name [[AS] alias [(column_alias [, ...])]`
   - Se sono presenti due o più tabelle, si esegue il prodotto cartesiano tra tutte le tabelle. Se ci sono attributi con lo stesso nome su tabelle diverse, nella SELECT questi attributi sono identificati anteponendo il nome della tabella e un ".": nomeTabella.nomeAttributo
2. `(other_select) [AS] nomeRisultato [(column_alias [,...])]`
   - è una SELECT innestata. Il risultato della SELECT interna è lo schema con nome nomeRisultato su cui fare la SELECT corrente.
3. `from_item [NATURAL] join_type from_item [ON condition]`
   - è la clausola JOIN.
   - Metodo più sofisticato che permette di selezionare un sottoinsieme del prodotto cartesiano di due o più tabelle. join_type specifica il tipo di join. I tipi principali sono riassunti nelle slide seguenti.

1.4 WHERE

```
WHERE condition
```

La clausola WHERE ammette come argomento un'espressione booleana costruita combinando predicati semplici con gli operatori AND, OR e NOT

- AND: seleziona le righe in cui tutti i predicati sono veri
- OR: seleziona le righe in cui almeno un predicato è vero
- NOT: seleziona le righe in cui il predicato è falso

Ciascun predicato semplice usa gli operatori =, <>, <, >, <= e >= p er confrontare un'espressione costruita a partire dal valore di un attributo con un valore costante, oppure un'altra espressione. Per esprimere la precedenza degli operatori AND e OR si fa uso della parentesi.

1.4.1 LIKE

Per il confronto tra stringhe, SQL mette a disposizione anche l'operatore LIKE che consente verificare se una stringa rispetta un certo "pattern" (oppure ILIKE per case insentive).
Due caratteri speciali per il confronto "_" e "%":

- "_": confronto con un carattere arbitrario
- "%": confronto con una stringa di lunghezza arbitraria (anche 0) di caratteri arbitrari

'_*o*%*i*' = stringa che ha una o in seconda posizione, seguita da una sequenza qualsiasi di caratteri (anche vuota) e che termina con una i.

1.4.2 BETWEEN

L'operatore BETWEEN consente di verificare se un valore è compreso tra due valori

```
WHERE attributo [NOT] BETWEEN valore1 AND valore2;
```

Il predicato ritorna true se il valore di attributo è compreso tra valore1 e valore2 (inclusi). Se è presente NOT, ritorna true se il valore di attributo è minore di valore1 o maggiore di valore2.

1.4.3 SIMILAR TO

L'operatore SIMILAR TO è un LIKE più espressivo che accetta, come pattern, un sottoinsieme delle espressioni regolari POSIX (versione SQL).
Esempi di componenti di espressioni regolari:

- _: confronto con un carattere arbitrario
- %: 0 o più caratteri arbitrari

- : ripetizione dell'elemento precedente (carattere o gruppo) 0 o più volte
- +: ripetizione dell'elemento precedente (carattere o gruppo) 1 o più volte
- $\{n, m\}$: ripetizione del precedente match almeno n e non più di m volte.
- [...]: elenco di caratteri ammissibili

1.4.4 IN

Nella clausola WHERE può apparire l'operatore [NOT] IN per testare l'appartenenza di un valore ad un insieme.

```
WHERE attributo [NOT] IN ( valore [, ...] );
```

1.4.5 IS NULL

Per verificare se un attributo ha un valore nullo o meno si utilizza il predicato IS NULL.

- Attributo IS NULL: ritorna true se l'attributo ha valore nullo
- IS NOT NULL: è la negazione, ritorna true se l'attributo ha valore diverso da nullo

```
WHERE attributo IS [ NOT ] NULL;
```

SQL-2 implementa la logica a 3 valori:

- Un predicato semplice restituisce il valore NULL quando uno qualsiasi dei termini del predicato ha valore nullo.
- Il predicato IS NULL è una eccezione, restituendo sempre il valore vero oppure false, mai il valore NULL.

1.5 SQL e Algebra Relazionale

1.5.1 Duplicati

Una significativa differenza tra SQL e algebra relazionale è la gestione dei duplicati. L'operazioni di eliminazione delle righe doppie (ad esempio al seguito di una selezione con where) è molto costosa, quindi di default SQL mantiene le righe duplicate.

- Il costrutto DISTINCT dopo la parola chiave SELECT permette di eliminare i duplicati.
- Il costrutto ALL permette invece di mantenerli, ma questa è già la scelta di default.

1.5.2 Selezione, Proiezione, Join

- L'istruzione SELECT, con una sola relazione nella clausola FROM permette di realizzare: selezioni, proiezioni, ridenominazioni.
- con più relazioni nella clausola FROM si realizzano join (e prodotti cartesiani)

In SQL il join si può definire in due modi diversi:

- Il primo non richiede l'uso di nuova sintassi, e specifica il legame tra le tabelle coinvolte a livello della clausola WHERE.
- Il secondo usa una sintassi riservata a livello di clausola FROM (useremo questo).

La clausola FROM ammette come argomento: `table_name [[AS] name]] [, ...]`. Si è visto che se sono presenti due o più nomi di tabelle, si esegue il prodotto cartesiano tra tutte le tabelle e lo schema del risultato può contenere tutti gli attributi del prodotto cartesiano. Il prodotto cartesiano di due o più tabelle è un **CROSS JOIN**. A partire da SQL-2, esistono altri tipi di JOIN (join_type): INNER JOIN, LEFT [OUTER] JOIN, RIGHT [OUTER] JOIN e FULL [OUTER] JOIN.

1.6 JOIN

```
table_name [ NATURAL ] join_type table_name [ON join_condition [, ...]].
```

dove join_condition è un'espressione booleana che seleziona le tuple del join da aggiungere al risultato. Le tuple selezionate possono essere poi filtrate con la condizione della clausola WHERE. Il join si può spostare nella clausola from, a livello delle tabelle (nello svolgere gli esercizi, useremo questa sintassi dedicata).

- ```
  SELECT ListaAttributi
  FROM Tabella1 JOIN Tabella2 ON ( CondizioneDiJoin )
  [ WHERE AltraCondizione]
  ```

- ```
  SELECT i.Nome, d.Città
  FROM Impiegato AS i JOIN Dipartimento as d ON ( i.Dipart = d.Nome )
  ```

Le parole OUTER, LEFT, RIGHT e FULL sono opzionali. Se non sono presenti, si intende INNER JOIN.

1.6.1 INNER JOIN

```
table1 [ INNER ] JOIN table2 ON join_condition [, ...].
```

Rappresenta il tradizionale θ-join dell'algebra relazionale. Combina ciascuna riga r1 di table1 con ciascuna riga di table2 che soddisfa la condizione della clausola ON.

1.6.2 LEFT [OUTER] JOIN

```
table1 LEFT [ OUTER ] JOIN table2 ON join_condition [, ...].
```

Si esegue un INNER JOIN. Poi, per ciascuna riga r1 di table1 che non soddisfa la condizione con qualsiasi riga di table2, si aggiunge una riga al risultato con i valori di r1 e assegnando NULL agli altri attributi...

> **Warning 1.1** (Attenzione)
> Il LEFT [OUTER] JOIN non è simmetrico! Con le medisime tabelle si possono ottenere risultati diversi invertendo l'ordine delle tabelle nel join!

1.6.3 RIGHT [OUTER] JOIN

```
table1 RIGHT [ OUTER ] JOIN table2 ON join_condition [, ...].
```

Si esegue un INNER JOIN. Poi, per ciascuna riga r2 di table2 che non soddisfa la condizione con qualsiasi riga di table1, si aggiunge una riga al risultato con i valori di r2 e assegnando NULL agli altri attributi..

1.6.4 FULL [OUTER] JOIN

```
table1 FULL [ OUTER ] JOIN table2 ON join_condition [, ...].
```

È equivalente a: INNER JOIN + LEFT OUTER JOIN + RIGHT OUTER JOIN.

1.7 ALIAS

SQL permette di associare un nome alternativo (alias) alle tabelle che compaiono nella clausola FROM. Il nome viene usato per far riferimento alla tabella nel contesto dell'interrogazione. Questo perché:

- Far riferimento alla tabella in modo compatto.
- Fare accesso più volte alla stessa tabella

1.8 ORDER BY

Come abbiamo visto, una relazione è un insieme di tuple non ordinato. Nella realtà, però, sorge spesso il bisogno di costruire un ordine sulle righe. SQL permette di specificare un ordinamento sulle righe dell'output grazie alla clausola ORDER BY.

```
ORDER BY attributo [{ ASC | DESC }] [, ... ];
```

Le righe vengono ordinate per il primo attributo nell'elenco, a parità di valore del primo attributo si usa il secondo, e così via in sequenza.

1.9 OPERAZIONI INSIEMISTICHE

SQL mette a disposizione degli operatori insiemistici.

```
SELECT
{ UNION | INTERSECT| EXCEPT
[ALL] SELECT }
```

Gli operatori insiemistici, al contrario del resto del linguaggio, assumono come default di eseguire un'eliminazione dei duplicati, a meno che si usi ALL.
Sono sufficienti domini compatibili ed ugual numero di attributi

- Ordine posizionale: la corrispondenza degli attributi si basa sulla posizione
- Se i nomi degli attributi sono diversi di solito il sistema usa il nome del primo operando

1.10 OPERATORI AGGREGATI

Una delle più importanti estensioni di SQL rispetto all'algebra relazionale: **COUNT, SUM, AVG, MIN, MAX**. In algebra relazionale tutte le condizioni specificate vengono valutate su una tupla alla volta. Gli operatori di aggregazione permettono di valutare proprietà che dipendono da insiemi di tuple. Prima viene eseguita l'interrogazione considerando solo le clausole FROM e WHERE, quindi l'operatore di aggregazione viene applicato alla tabella contenente il risultato.

1.10.1 COUNT

L'operatore COUNT conta il numero di righe di una tabella o il numero di valori di uno o più attributi:

```
COUNT ( <* | [DISTNCT | ALL] ListaAttributi> )
```

- *: conta il numero di righe della tabella
- DISTINCT ListaAttributi: conta il numero di valori diversi degli attributi ListaAttributi
- ALL ListaAttributi: conta il numero di righe che hanno valori diversi da NULL negli attributi in ListaAttributi (comportamento di default)

1.10.2 MAX, MIN, AVG, SUM

```
<SUM | MAX | MIN | AVG > ( [DISTINCT | ALL] AttrExpr )
```

- SUM: calcola la somma dei valori di AttrExpr
- MAX: calcola il valore massimo di AttrExpr

- MIN: calcola il valore minimo di AttrExpr
- AVG: calcola la media dei valori di AttrExpr

SUM e **AVG** ammettono come argomento solo espressioni che rappresentano valori numerici o intervalli di tempo. **MAX** e **MIN** ammettono come argomento solo espressioni su cui è definito un ordinamento (quindi anche stringhe). **DISTINCT** elimina i duplicati mentre **ALL** trascura solo i valori NULL. (DISTINCT e ALL non hanno effetto su MIN e MAX)

```
SELECT Cognone, Nome, MAX( I.Stipendio )
FROM Impiegato as I JOIN Diparmento as D
ON I.Dipart = D.Nome
WHERE D.Città = 'Milano'
```

Warning 1.2

la precedente query non è corretta! Gli operatori aggregati non sono un meccanismo di selezione, ma solo funzioni che restituiscono un valore quando sono applicate ad un insieme. Cognome e Nome restituiscono un valore per ogni tupla selezionata, mentre MAX() restituisce un valore unico per tutto l'insieme. Per risolvere questa situazione serve il meccanismo di raggruppamento che non vediamo.

1.11 OPERATORI DI RAGGRUPPAMENTO

1.11.1 GROUP BY

Gli operatori aggregate possono essere applicati separatamente a sottoinsiemi di righe. La clausola GROUP BY permette di specificare come dividere le tabelle in sottoinsiemi. La clausola permette di specificare un insieme di attributi e l'interrogazione raggrupperà le righe che possiedono gli stessi valori per questo insieme di attributi. In una interrogazione che fa uso della clausola GROUP BY può comparire come argomento della SELECT solo un sottoinsieme degli attributi usati nella clausola GROUP BY oppure operatori di aggregazione.

1.11.2 HAVING

È possibile applicare delle selezioni ai gruppi generati tramite la clausola GROUP BY. La clausola HAVING permette di specificare delle condizioni che devono essere soddisfatte dai gruppi generati dopo l'esecuzione del raggruppamento tramite GROUP BY.

Osservazione 1.1

- La clausola WHERE applica delle selezioni su SINGOLE RIGHE, prima dell'eventuale raggruppamento tramite GROUP BY.

- La clausola HAVING applica delle selezioni su GRUPPI, generati a seguito dell'esecuzione del raggruppamento tramite GROUP BY.

1. WHERE → seleziona le righe che verranno raggruppate
2. GROUP BY → raggruppa le righe selezionate
3. HAVING → seleziona i gruppi che fanno parte del risultato

1.12 Interrogazioni nidificate

Una interrogazione nidificata è una interrogazione che contiene al suo interno altre interrogazioni. La nidificazione può avvenire sia nella clausola SELECT, che nella clausola FROM, che nella clausola WHERE. Il caso tipico è la nidificazione nella clausola WHERE, dove un predicato contiene un confronto tra un valore (espressione su una singola riga) ed il risultato di un'interrogazione SQL.

1.12.1 Nidificazione nella clausola WHERE

Se in un predicato si confronta un attributo o una espressione che produce un valore singolo con il risultato di un'interrogazione, sorge il problema di disomogeneità dei termini del confronto. I normali operatori di confronto ($=, <>, <, >, <=, =>$) possono essere estesi con le parole chiave ANY e ALL.

- ANY: la riga soddisfa la condizione se risulta vero il confronto (con l'operatore specificato) tra il valore dell'attributo (o espressione) e **ALMENO** uno degli elementi restituiti dall'interrogazione.
- ALL: la riga soddisfa la condizione se risulta vero il confronto (con l'operatore specificato) tra il valore dell'attributo (o espression) e **TUTTI** gli elementi restituiti dall'interrogazione

Per le interrogazioni nidificate viste fino ad ora, l'interrogazione può essere eseguita una volta sola ed il risultato ottenuto può essere salvato in una tabella temporanea usata per eseguire ciascun confronto con le righe dalla tabella esterna. Talvolta però l'interrogazione nidificata fa riferimento al contesto dell'interrogazione che la racchiude attraverso l'uso di una variabile (*passaggio di binding*). L'interrogazione nidificata dovrà essere valutata separatamente per ciascuna riga prodotta dalla valutazione della query esterna.

1.12.2 Nidificazione nidifiicate Binding e EXISTS

L'operatore logico **EXISTS** riceve come parametro un'interrogazione nidificata e restituisce il valore vero solo se l'interrogazione fornisce un risultato non vuoto. Questo operatore può essere usato in modo significativo solo quando si ha un passaggio di binding tra la query esterna e quella interna.

```
SELECT * FROM Persona P
WHERE EXISTS (SELECT * FROM Persona P1 WHERE P1.Nome
    = P.Nome AND P1.Cognome = P.Cognome AND P1.CodFiscale
    <> P.CodFiscale );
```

Nel codice sopra si estraggono le persone che hanno degli omonimi (ovvero persone con lo stesso nome e cognome, ma codice fiscale diverso). In questo prossimo codice si vanno ad estrarre le persone che non hanno degli omonimi (ovvero persone con lo stesso nome e cognome, ma codice fiscale diverso)

```
SELECT * FROM Persona P
WHERE NOT EXISTS (SELECT * FROM Persona P1
    WHERE P1.Nome = P.Nome AND
    P1.Cognome = P.Cognome AND
    P1.CodFiscale <> P.CodFiscale );
```

Le interrogazioni nidificate possono essere specificate anche come elemento della clausola SELECT. Spesso l'uso di query nidificate nella SELECT sostituiscono l'uso del JOIN. Per esempio, estrarre per ogni cantante il numero di canzoni di cui è autore.

```
SELECT DISTINCT Nome, NumCanzoni =
    (SELECT COUNT(*) FROM Autore A
    WHERE A.Nome = C.Nome)
    FROM Cantante C
```

oppure

```
SELECT DISTINCT C.Nome, COUNT(*) as NumCanzoni
FROM Cantante C JOIN Autore A
ON C.Nome = A.Nome
GROUP BY C.Nome
```

Osservazione 1.2

Ricordiamo che è necessario che l'interrogazione nidificate restituisca una sola tupla (riferimenti tramite variabile alla interrogazione esterna).

Le interrogazioni nidificate possono essere utilizzate nella clausola FROM come sorgente dati su cui eseguire l'interrogazione. È possibile creare una catena di interrogazioni, dove ciascuna interrogazione utilizza il risultato della precedente come una tabella di base. Per esempio: si estrae per ogni cantante che è anche autore di canzoni il numero di canzoni di cui è rispettivamente interprete e autore.

```
SELECT DISTINCT C.Nome, C.NumCantate, A.NumScritte
FROM
(SELECT Nome, Count(*) as NumCantate FROM Cantante) C
JOIN
(SELECT Nome, Count(*) as NumScritte FROM Autore ) A
ON C.Nome = A.Nome
```

Prima vengono eseguite le interrogazioni nella clausola FROM e i risultati di queste vengono usati come normali tabelle.

1.13 VIEW

Le viste permettono di salvare una query che genera una tabella temporanea.

```sql
CREATE VIEW <nome_vista> AS (<QUERY>)
```

Dato un nome alla query, la possiamo usare come se fosse il nome di una tabella.

1.14 Esercizi

Esempio 1.1

We have here three tables:

- **TURISTA** con gli attributi USERNAME, COGNOME, NOME, DATA_NASCITA, NAZIONALITA
- **ATTRAZIONE** con gli attributi CODICE, NOME, DESCRIZIONE, LATITUDINE, LONGITUDINE, INDIRIZZO, CITTA, PROV
- **RECENSIONE** con gli attributi ATTRAZIONE, TURISTA, TIMESTAMP, TITOLO, TESTO, VOTO

1. Trovare tutte le attrazioni della città di Verona il cui nome inizia con la lettera *f* maiuscola o minuscola.

```sql
SELECT *
FROM ATTRAZIONE
WHERE CITTA = 'Verona' AND NOME ILIKE 'f%';
```

2. Trovare tutti i turisti che nel 2025 non hanno mai rilasciato recensioni con un voto > 2 per attrazioni fuori dalla provincia di Verona.

```sql
SELECT T.USERNAME FROM TURIST T
EXCEPT (
    SELECT R.TURISTA
    FROM RECENSIONE R JOIN ATTRAZIONE A ON R.ATTRAZIONE = A.CODICE
```

```sql
    WHERE
        R.VOTO > 2 AND
        A.PROVINCIA <> 'VERONA' AND
        EXTRACT (YEAR FROM R.TIMESTAMP) = '2025'
)
```

Osservazione 1.3

Le due query devono essere compatibili.

Questa query nidificata che restituisce i turisti che hanno rilasciato recensioni con voto > 2 per attrazioni fuori dalla provincia di Verona nel 2025 e la chiararemo Q1. Oppure

```sql
SELECT T.USERNAME, T.NOME, T.COGNOME FROM TURIST T
NOT IN (
    SELECT R.TURISTA
    FROM RECENSIONE R JOIN ATTRAZIONE A ON R.ATTRAZIONE = A.CODICE
    WHERE
        R.VOTO > 2 AND
        A.PROVINCIA <> 'VERONA' AND
        EXTRACT (YEAR FROM R.TIMESTAMP) = '2025'
)
```

Osservazione 1.4 (Q)

esta query non richiede che gli schemi siano compatibili,e permette di aggiungere facilmente altri attributi alla query esterna

3. Trovare tutte le attrazioni che hanno ricevuto nel 2025 un numero di recensioni superiore a quelle ricevute nel 2024.

   - Date tutte le righe di R1
   - Le divide per attrazioni in un gruppo per ogni ATTRAZIONE
   - Applica count su ogni gruppo

```sql
SELECT COUNT(*) AS NumRecensioni, R1.ATTRAZIONE
FROM RECENSIONE R1
WHERE EXTRACT (YEAR FROM R1.TIMESTAMP) = '2025'
GROUP BY R1.ATTRAZIONE
HAVING
    NumRecensioni > (
        SELECT COUNT(*) AS NumRecensioni2024
        FROM RECENSIONE R2
        WHERE
```

```
            EXTRACT (YEAR FROM R2.TIMESTAMP) = '2024' AND
            R1.ATTRAZIONE = R2.ATTRAZIONE
    );
```

Nella query annidata non ci sta bisogno di fare il `GROUP BY` poiché abbiamo già fatto il DATA BINDING tra R1 e R2 con `R1.ATTRAZIONE = R2.ATTRAZIONE` e di conseguenza R2.ATTRAZIONE è già un valore singolo e non un insieme di valori.

4. Trovare la media dei voti massimi rilasciati dagli utenti.

   - Calcolare il voto massimo rilasciato da ciascun utente $\{max_1, \ldots, max_n\}$
   - Calcolare la media dei valori in $\{max_1, \ldots, max_n\}$

   Warning 1.3

   NO! Questa query non si può fare

```
SELECT AVG(MAX(VOTO))
...
```

   Procediamo in questa maniera:

```
SELECT TURISTA, MAX(VOTO) AS MaxVoto
FROM RECENSIONE
GROUP BY TURISTA
```

   e poi facciamo la media:

```
SELECT AVG(T.MaxVoto)
FROM (
    -- Prendiamo il voto massimo rilasciato da ciascun utente
    SELECT TURISTA, MAX(VOTO) AS MaxVoto
    FROM RECENSIONE R
    GROUP BY R.TURISTA
) AS T
```

Esempio 1.2

Consideriamo le seguenti tre tabelle:

- AUTO(TARGA, NUM_TELAIO, MODELLO, COLORE, KM, CILINDRATO, POSTI, PROPRIETARIO)
- PARCHEGGIO(NUMERO, COMUNE, INDIRIZZO, COPERTO)
- PRENOTAZIONE(CODICE, PARCHEGGIO, POSTO, AUTO,

DATA_ORA_INIZIO, DATA_ORA_FINE, TIPO, PAGATO)

Dove

$$\text{PRENOTAZIONE.PARCHEGGIO} \rightarrow \text{PARCHEGGIO}$$

$$\text{PRENOTAZIONE.AUTO} \rightarrow \text{AUTO}$$

Osservazione 1.5

Ricordiamo che SQL è case insentive.

1. Trovare tutti i parcheggi coperti di Verona

```sql
SELECT *
FROM PARCHEGGIO
WHERE COMUNE = 'VERONA' AND COPERTO;
```

2. Trovare tutti i parcheggi in cui non sono mai sstate fatte prenotazioni online nel 2024. Per ciascun parcheggio riportarare il nome ed il numero di prenotazioni complessive.

```sql
SELECT P.NOME, COUNT(*) AS NumeroPrenotazioni
FROM PARCHEGGIO P JOIN PRENOTAZIONE R ON (P.CODICE = R.PARCHEGGIO)
WHERE P.CODICE NOT IN (
    -- Query annidata che trova tutti i parcheggi che hanno avuto
    prenotazioni online nel 2024
    SELECT R1.PARCHEGGIO
    FROM PRENOTAZIONI R1
    WHERE
        EXTRACT (YEAR FROM DATA_ORA_INIZIO) = '2024' AND
        TIPO = 'ONLINE'
)
GROUP BY P.NOME;
```

3. Trovare tutti i parcheggi in cui vengono effettuate più prenotazioni online che prenotazioni fisiche.

```sql
SELECT PR.PARCHEGGIO
FROM PRENOTAZIONE PR
WHERE R.TIPO = 'ONLINE'
GROUP BY R.PARCHEGGIO
HAVING COUNT(*) > (
    SELECT COUNT(*)
    FROM PRENOTAZIONE R2
    WHERE
        R2.TIPO = 'FISICA' AND
        R2.PARCHEGGIO = R.PARCHEGGIO -- DATA BINDING
)
```

Osservazione 1.6

La query interna è eseguita per ogni riga della query esterna.

4. Trovare quanti parcheggi del comune di Verona hanno avuto un numoro medio di prenotazioni nel 2024 superiore al numero medio di prenotazioni complessive del 2024 dei parcheggi del comune di Padova.

   (a) Calcolare # prenotazioni nel 2025 per parcheggio per ogni parcheggio di Verona

   (b) Calcolare # prenotazioni nel 2025 per parcheggio per ogni parcheggio di Padova

   (c) Calcolare la media su (b)

```sql
CREATE VIEW NPRENOTAZIONI AS
(
    SELECT R.PARCHEGGIO, P.COMUNE, COUNT(*) AS NUM_PR
    FROM PARCHEGGIO P JOIN PRENOTAZIONE R ON (P.CODICE = R.PARCHEGGIO)
    WHERE EXTRACT (YEAR FROM R.DATA) = '2024'
    GROUP BY R.PARCHEGGIO, P.COMUNE
)

SELECT N.PARCHEGGIO
FROM NPRENOTAZIONI AS N
WHERE
    N.NUM_PR > (
        -- numero medio di prenotazioni dei parcheggi di PD nel 2024
        -- (ritorna un solo valore)
        SELECT AVG(N1.NUM_PR)
        FROM NPRENOTAZIONI AS N1
        WHERE N1.COMUNE = 'PADOVA'
    ) AND
    N.COMUNE = 'VERONA'; -- Per ogni parcheggio VR guardo quali hanno #
     prenotazioni maggiori
```

2 Metodi di accesso

2.1 Operazioni sulle pagine

- **Inserimento**
  - Se esiste uno spazio contiguo sufficiente ⇒ siamo nel caso semplice perché ci basta aggiungere una tupla e aggiornare il dizionario.
  - Se non esiste uno spazio sufficiente ⇒ dobbiamo avere un'interazione con il filesyste per allocare nuovi blocchi per i file.

  - Se esiste uno spazio sufficiente ma non contiguo $\Rightarrow$ Riorganizzazione spazio: inserimento semplice.

- **Cancellazione**
  - Sempre possibile e non necessita riorganizzazione

- **Accesso**
  - Tupla $\rightsquigarrow$ dizionario, accesso sequenziale
  - Ad un attributo di una tupla:
    * Identificare la tupla t
    * Scorrere elementi di t tramite offset

2.2 Criteri di organizzazione

I criteri per cui organizziamo le tuple nei blocchi possono essere:

- **Sequenziale**: abbiamo una disposizione consecutiva delle tuple in base ad un criterio specifico (es. ordine di inserimento).
- **Accesso calcolato (Hash)**: La posizione delle tuple nei blocchi dipende da un algoritmo di hashing.
- **Albero**: principato utilizzato per la costruzione di indici.

2.2.1 Struttura primaria sequenziale/seriale

Nella struttura primaria sequenziale, costruiamo i blocchi "logicamente" consecutivi. Assumiamo che questi blocchi siano uno vicini all'altro, in modo da minimizzare i tempi di accesso. Ci sono due modi per organizzare le tuple nei blocchi:

- **Disordinata**: le tuple sono disposte rispetto all'ordine di inserimento. È la forma più semplice con cui salviamo i nostri dati. Quali sono le sue caratteristiche?
  - **Inserimento:** prendo l'ultimo blocco (b) del file libero e tento di fare l'inserimento b. Ha bisogno di un solo accesso a memoria secondaria.
  - **Ricerca:** non è efficiente perché dovremmo fare una scansione sequenziale di tutti i blocchi del file.
  - **Cancellazione/Modifica:** Per potere eliminare una tupla abbiamo prima bisogno di trovarla. Dunque richiede la ricerca. Una volta che l'abbiamo trovata, basta semplicemente che cancelliamo senza ricompattare. Nella modifica invece può cambiare la dimensione della tepla e potrebbe non starci nello stesso spazio. In questo caso, devo cancellare e inserire. Per far fronte a questo problema, il DBMS può riorganizzare i blocchi periodicamente.

- **Ordinata**: le tuple sono disposte rispetto ad un campo chiamato "chiave di ordinamento". Questa struttura rende efficiente le operazioni che hanno bisogno dell'ordinamento utilizzato ad esempio:
  - Elenco ordinato
  - Range query
  - Operazioni aggregate

  In caso di aggiornammento è necessario mantenere l'ordinamento, quindi servono periodiche riorganizzazioni dei blocchi.
  - **Inserimento:**
    * Quando devo fare l'inserimento devo prima individuare il blocco in cui fare l'inserimento. Devo andare a pescare il blocco candidato ovvero quello che contiene la tupla precedente nell'ordinamento.
    * Se c'è spazio, allora faccio l'inserimento e basta, altrimenti devo aggiungere un nuovo blocco in cui mettere questa tupla.
  - **Cancellazione:**
    * Devo individuare il blocco b che la contiene
    * Cancellare la tupla
    * Aggiusto i puntatori tra i blocchi per mantenere l'ordinamento
  - **Riorganizzazione:** TODO

2.3 Indici

Gli indici sono delle strutture dati per l'accesso efficiente ai file sequenziali. L'obiettivo degli indici è quello di velocizzare l'accesso casuale via chiave di ricerca (un insieme di attributi). Gli indici si contraddistinguono tra due tipi:

- **Indici primari**: la chiave di ordinamento è uguale alla chiave di ricerca. Ogni record dell'indice è composto da:

  $$< v_i, p_i >$$

  dove v_i è il valore della chiave di ricerca e p_i è il puntatore alla prima tupla con quel valore di chiave (basta 1 puntatore perché chiave ricerca = chiave ordinamento). Trovata la prima tupla, le altre sono di seguito. Inoltre l'indice può essere
  - **denso**, ovvero che esiste un recordo di indice per ogni chiave di ricerca.
  - **sparso**, solo per alcune occorrenze dlle chiave persenti nel file esiste un corrispondente record nell'indice, tipicamente una per blocco.

- Per la **ricerca** di una chiave k se è **denso**:
  * K è presente nell'indice? Se no, allora è finito subito e ho solo un acccesso all'indice (vuol dire fare una scansione sequenziale dei blocchi in cui è salvato l'indice). Altrimenti, se è presente, faccio accesso al blocco puntato da p_k e quindi il mio costo equivale a un accesso all'indice + un accesso al blocco dati. Per l'accesso all'indice devo comunque caricar el'indice di memoria che sono comunque molto meno rispetto a fare una scansione sequenziale.
- Per la **ricerca** di una chiave K se è **sparso** (k potrebbe non essere presente nell'indice ma essere presente nelle tuple.):
  * Eseguo una scansione sequenziale dell'indice per trovare (k', p_k) dove k' è il valore di chiave più grande minore o uguale alla chiave k che sto cercando
  * A questo punto seguo il puntatore p_k per accedere al blocco. In questo caso il costo dell'operazione è:

$$\text{costo} = \text{un accesso all'indice} + \text{un accesso al blocco dei dati.}$$

- **Inserimento**:
  * Come un inserimento su file sequenziale disordinato/ordinato
  * Aggiornare indice (mantenere l'indice è costoso, quindi bisogna fare dei buon compromessi su il numero di indici che costruiamo e il numero di attributi su cui non calcoliamo l'indice poiché non ha senso)
    · Se è denso, inserire una coppia nel file indice solo se la chiave è nuova
    · Se è sparso, inserire una coppia solo se l'inserimento della tupla aggiunge un nuovo blocco
- **Cancellazione**:
  * Cancellazione dal file sequenziale disordinato/ordinato
  * Aggiorniamo l'indice:
    · Se è denso, cancelliamo anche la coppia nel file di indice solo se la tupla era l'ultima con chiave k.
    · Se è sparso procediamo a cancellare anche la coppia nel file di indice se la tupla era l'ultima con chiave k inoltre aggiungere una nuova coppia con chiave k' (chiave più grande minore o uguale a k) se k' è diverso da k.

- **Indici secondari**: la chiave di ordinamento è diversa dalla chiave di ricerca. Ogni record dell'indice è composto da:

$$< v_i, p_i >$$

dove v_i è il valore della chiave di ricerca e p_i è un puntatore ad un bucket di puntatori che individuano nel file sequenziale tutte le tuple con chiave v_i. Gli indice secondari **sono**

sempre densi perché non si può sfruttare l'ordinamento delle tuple.
Le operazioni che si possono eseguire su un indice secondario sono:

- **Ricerca** di una chiave k:
  - Ricerca sequenziale per trovare $k \Rightarrow (k, p_k)$:
    - Se è denso, se k non c'è, non esistono tuple con chiave k
  - Accesso al bucket b p_k
  - Accesso al file tramite i puntatori in b.

  Il costo dell'operazione è:

  $$\text{costo} = \text{un accesso all'indice} + \text{un accesso al bucket} + n \text{ accessi al file}$$

  dove n è uguale al numero di puntatori in b.

- **Inserimento/Cancellazione**: rimane come un indice primario denso ma con aggiornamento dei bucket.

2.4 Minimizzare i costi di accesso

Per migliorare l'accesso agli indici, possiamo migliorare le prestazioni utilizzando la struttura ad albero (B+TREE) oppure le strutture ad accesso calcolato (Hash).

2.4.1 B+TREE

- Struttura ad albero
- Ogni nodo viene memorizzato in una pagina/blocco di memoria secondaria.
- Ogni nodo ci sono dei legami che rappresentano i puntatori alle pagine.
- È caratterizzato per avere pochi livelli e molti nodi foglia.
- È un albero bilanciato in modo che non esistono dei path preferenziali o più "fortunati" rispetto agli altri. Inserimenti e cancellazioni non devono alterare le prestazioni di accesso ai dati.

Struttura dell'albero

- **Nodi foglia:** (con fan-out = n) Può contenere fino a $n - 1$ valori ordinati di chiave di ricerca e fino ad n puntatori ai dati.

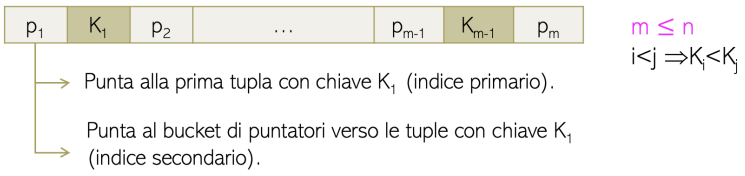


Figure 1: Struttura di un B+TREE con fan-out = n

Ogni nodo foglia ha un *vincolo di ordinamento*. Questo vuol dire che le chiave dentro ad un nodo foglia sono ordinate e inoltre sono ordinate rispetto alle chiavi degli altri nodi foglia.

$$\forall L_i, L_j \text{ nodi foglia } i < j : \forall k_t \in L_i, k_j \in L_j : k_i < k_j$$

Il puntatore p_m del nodo L_i punta al nodo L_{i+1}, ovvero al nodo foglia successivo se esiste e altrimenti è nullo.

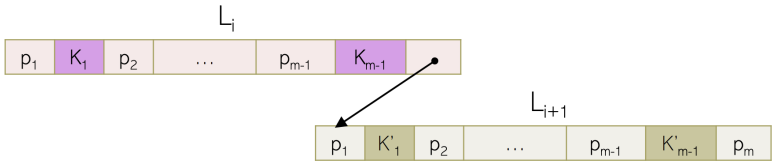


Figure 2: Puntatore p_m del nodo L_i che punta al nodo L_{i+1}

- **Nodi intermedi** (con fan-out = n) è una sequenza di $m' \leq n$ valori ordinati di chiave. Può contenere fino a n puntatori a nodo. Ogni chiave K_i è seguita da un puntatore p_i:

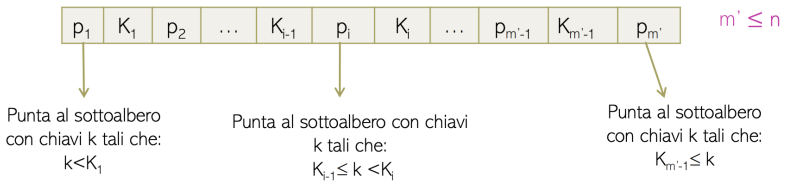


Figure 3: Struttura di un nodo intermedio di un B+TREE con fan-out = n

Entrambi i due nodi devono rispettare il vincolo di reimpimento.

- Per un nodo foglia con fanout = n, ogni nodo foglia può contenere un numero di valori di chiae (#chiavi) vincolato come:

$$\left\lceil \frac{n-1}{2} \right\rceil \leq \#\text{chiavi} \leq n - 1$$

- Per un nodo intermedio con fanout $= n$, ogni nodo intermedio contiene un numero di puntatori (#puntatori) vincolato come:

$$\left\lceil \frac{n}{2} \right\rceil \leq \#\text{puntatori} \leq n$$

Esempio 2.1

Esempio di B+Tree con contenuti (A, B, C, D, E, F, G, L, M, N, P) e con fanout = 4. Il fanout ci dice che ci sta un vincolo di reimpimento dove i nodi foglia devono essere:

$$2 \leq \ \#\text{chiavi} \leq 3$$

e i nodi intermedi devono essere:

$$2 \leq \ \#\text{puntatori} \leq 4$$

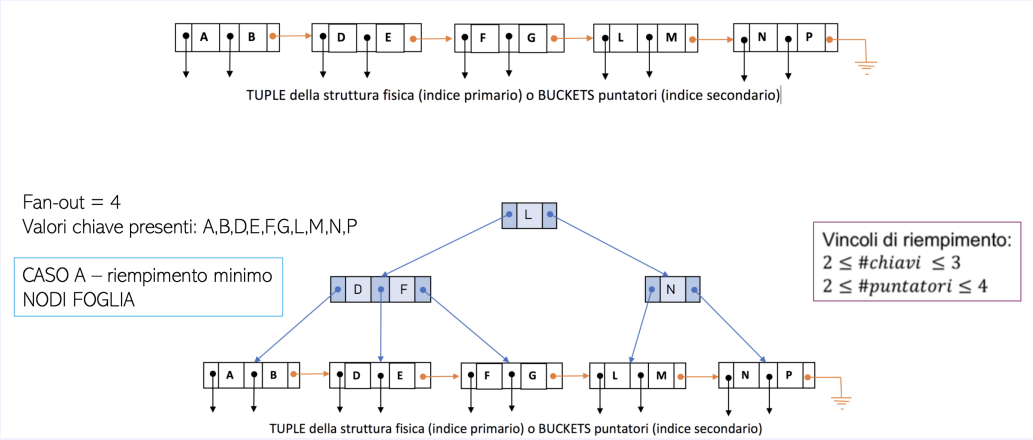


Figure 4: Esempio di B+Tree con fanout = 4

Questo esempio di albero per chiavi desiderate che rispetta il vincolo di reimpimento usando un reimpimento **minimo**. Tuttavia si poteva gestire il reimpimento in modo diverso, ad esempio con un reimpimento **massimo** che avrebbe portato ad avere nodi con 3 chiavi e 4 puntatori.

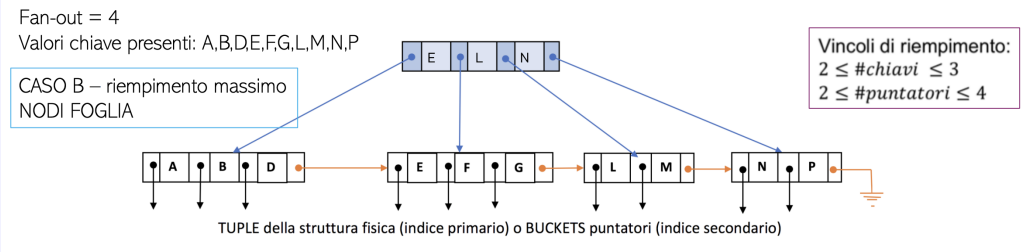


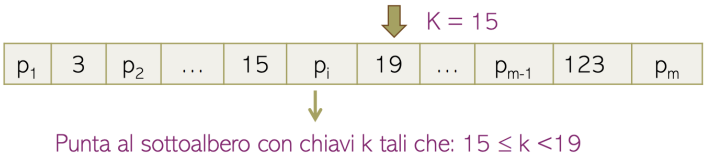
Figure 5: Esempio di B+Tree con fanout = 4 e reimpimento massimo

In questo caso non è necessario passare da un livello intermedio, possiamo rappresentare tutto con unico nodo radice, perché puntare a 9 nodi foglia ci servono 4 puntatori e questo è compatibile con il vincolo di reimpimento.

2.4.2 Ricerca in un B+TREE

1. Cercare nel nodo radice il più piccolo valore di chiave > di K.

   - Se tale valore esiste (supponiamo sia K_i) allora seguire il puntatore p_i.



   - Se tale valore non esiste seguire il puntatore p_m.

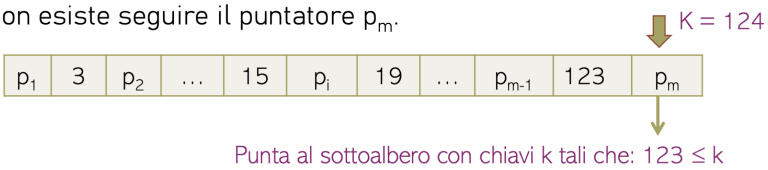
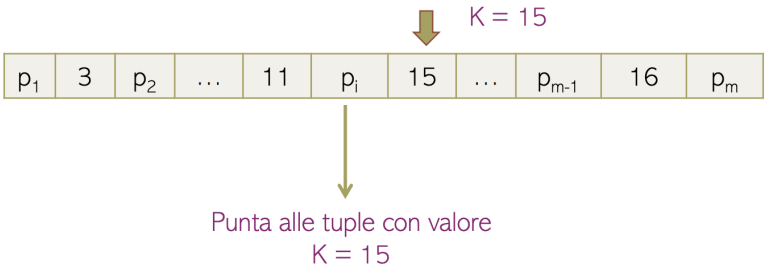


Figure 6: Ricerca di una chiave K in un B+TREE

2. Se il nodo raggiunto è un nodo foglia cercare il valore K nel nodo e seguire il corrispondente puntatore verso le tuple, altrimenti riprendere il passo 1



Esempio 2.2

Ricerca delle tuple con valore G della chiave di ricerca:

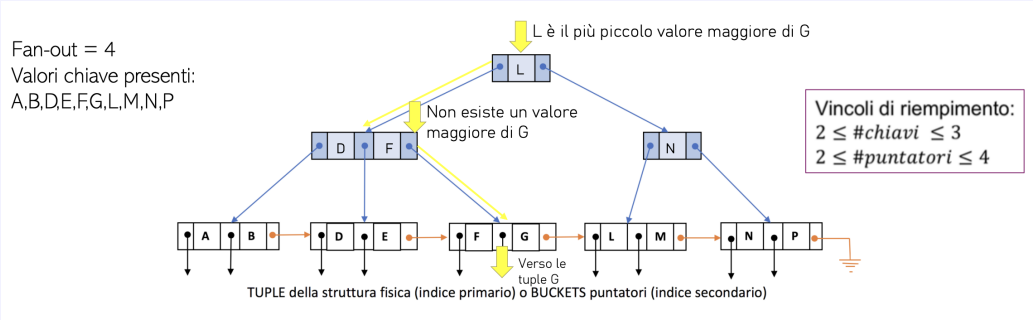


Figure 7: Esempio di ricerca di una chiave G in un B+TREE

Osservazione 2.1 (Costo di una ricerca)

l costo di una ricerca di un valore K in un B+TREE è dato da:

- La profondità dell'albero (ovvero il numero di nodi intermedi che dobbiamo attraversare) Questo dipende dal *fanout*. La profondità è limitata da:

$$prof_{B+Tree} \leq \underbrace{1}_{\text{radice}} + \underbrace{\log_{\lceil \frac{n}{2} \rceil} \underbrace{\left(\frac{\#\text{valoriChiave}}{\lceil \frac{n-1}{2} \rceil} \right)}_{\text{MAX \# FOGLIE}}}_{\text{\# livelli}}$$

- # di accessi alla memoria secondaria + ogni nodo è un blocco

2.4.3 Inserimento in un B+TREE

1. Ricerca del nodo foglia NF dove K va inserito.
2. Ci possono essere vari casi:

- Se K è presente in NF:
  - Se è un indice primario, allora nessuna azione.
  - Se è un indice secondario, allora aggiornare il bucket dei puntatori.
- Se K non è presente in NF, inserire K in NF rispettando ordine:
  - Se è un indice primario, inserire K e il puntatore alla tupla.
  - Se è un indice secondario, inserire K e creare un nuovo bucket che sarà puntato all'interno dell'indice.

Se violo il vincolo di riempimento massimo, devo dividere NF in due nodi foglia.

3. Passo split: se in NF esistono già n valori di chiave:
   - Creare due nodi foglia NF_1 e NF_2.
   - Devo inserire i primi $\lceil \frac{n-1}{2} \rceil$ valori di chiave in NF_1 e i restanti in NF_2.
   - Aggiornare il padre. Se il padre viola il vincolo di riempimento massimo, devo propagare lo split. Questa propagazione può arrivare fino alla radice e in questo caso devo creare un nuovo nodo radice.

Esempio 2.3

Consideriamo il seguente esempio:

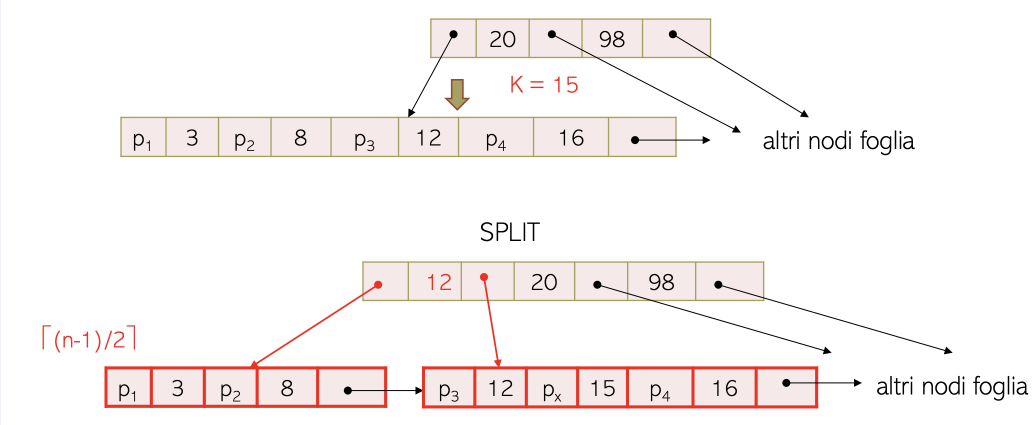


Figure 8: Esempio di split di un nodo foglia NF in un B+TREE con fanout = 5

Se aggiungo 15 a $NF \Rightarrow$ # chiavi = 5 allora violo il vincolo di riempimento massimo, devo fare lo split.

2.4.4 Cancellazione

1. Ricerca del nodo foglia NF che contiene K.

2. Cancellare K ed il suo puntatore:

   - Potrebbe violare il vincolo di riempimento minimo. Dopo la cancellazione:

$$\#\text{chiavi} < \left\lceil \frac{n-1}{2} \right\rceil$$

     In questo caso dobbiamo ricorrere all'operazione opposta dello split: il **merge**. Per effettuare il merge:

     - Individuiamo il nodo fratello.
     - Si genera un unico nodo foglia. Se non esiste il fratello che unito rispetta il vincolo di riempimento max, merge + split.
     - Si aggiorna il nodo padre
     - Se anche il padre viola il vincolo di riempimento minimo, merge anche per il padre.

Esempio 2.4

Consideriamo il seguente esempio di merge:

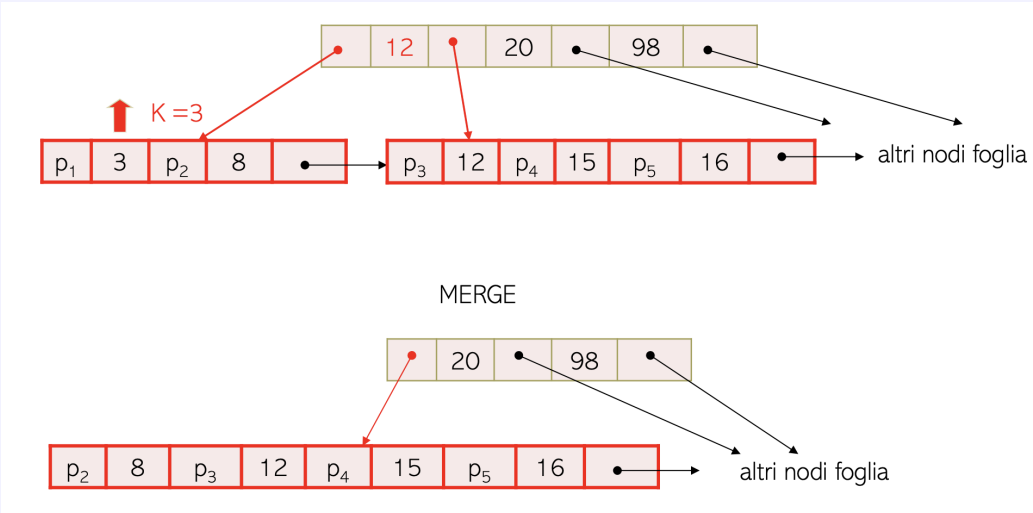


Figure 9: Esempio di merge di un nodo foglia NF in un B+TREE con fanout = 5

Dopo eliminazione di K uguale a 3, il nodo NF_1 viola il vincolo di reimpimento minimo.

Esempio 2.5 (Esercizio da Esame)

Costruire B+Tree con fanout = 5 con i seguenti valori di chiave:

$$(A, B, C, D) \quad (F, G, M, N) \quad (O, P) \quad (S, T) \quad (W, Z)$$

Vincoli di reimpimento:

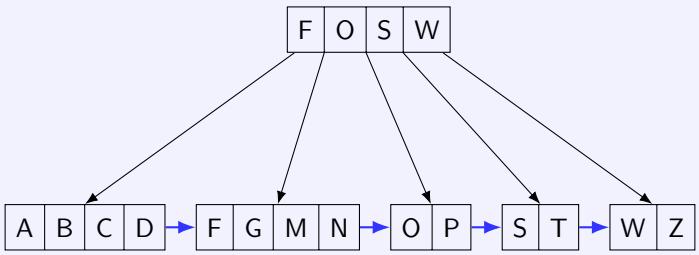
$$NF : \left\lceil \frac{n-1}{2} \right\rceil = 2 \leq \ \#\text{chiavi} \leq 4 = n - 1$$

$$NI : \left\lceil \frac{n}{2} \right\rceil = 3 \leq \ \#\text{puntatori} \leq 5 = n$$

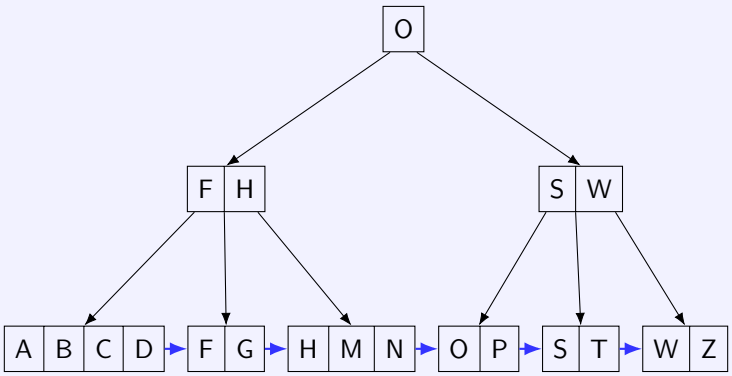
Le task sono le seguenti:

1. Costruzione dell'albero
2. Inserire H
3. Cancellare Z

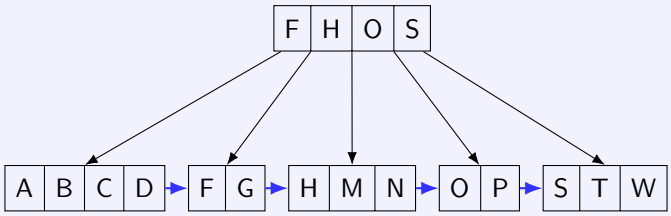
(1) Abbiamo 5 nodi foglia quindi abbiamo bisogno di 5 puntatori. Possiamo usare un unico livello intermedio.



(2) Inseriamo H: Partendo dalla radice si cerca il nodo più piccolo con chiave maggiore di H che in questo caso è O. Ci sta un problema però. Il nuovo nodo foglia viola il vincolo di riempimento massimo. Dobbiamo ricorrere allo split del nodo foglia NF che contiene F, G, H, M, N. Facendo però abbiamo 6 puntatori e questo viola il vincolo di riempimento massimo dei nodi intermedi. Quindi non possiamo più usare un unico livello intermedio, ma dobbiamo aggiungere un nuovo livello intermedio.



(3) Cencelliamo ora il nodo Z. Si cerca la chiave, non esiste quindi si seuge il puntatore a destra. Sappiamo però che se lo cancello, il nodo foglia NF che contiene W e Z viola il vincolo di riempimento minimo. Il nodo foglia ha un unico fratello e quindi senza ambiguità riesco a fonderlo con il nodo S, T.



Ora che abbiamo cancellato Z, abbiamo di nuovo 5 puntatori quindi possiamo usare un unico livello intermedio, come prima. Quindi viene creata una nuova radice.

2.4.5 Hash

Questa struttura sfrutta l'accesso associativo nel senso che la localizzazione fisica dei dati diende dal valore della chiave. Viene utilizzata una funzione di hash che mappa i valori di chiave in indirizzi di memorizzazione delle tuple.

$$h : K \to B$$

dove K è il dominio delle chiavi e B è il dominio degli indirizzi. Costo della ricerca è uguale quindi ad un accesso alla memoria secondaria. Tuttavia il problema dell'hashing sono le collisioni: valori di chiavi diverse possono produrre lo stesso indirizzo. Una funzione di hash è buona se minimizza il numero di collisioni. Viene definito il concetto di fattore di riempimento.

Definizione 2.1

Il fattore di riempimento mi dice lo spazio medio fisico utilizato in ciascun blocco.

Consideriamo:

- $T = \#\text{tuple}$
- $F = \#\text{tuple ci stanno in un blocco}$
- $f =$ fattore di riempimento
- $B = \left\lceil \frac{T}{F \cdot f} \right\rceil$ viene allocato il numero di bucket di puntatori uguale a B.

Viene definita una funzione di folding:

$$\varphi : K \to \mathbb{Z}^+ \text{ trasforma chiavi in interi}$$

La funzione di hashing è data da:

$$h : \mathbb{Z}^+ \to B$$

2.4.6 Ricerca in un indice hash

1. Dobbiamo calcolare $b = h(\varphi(K))$ dove il costo è dato da un accesso alla memoria secondaria e quindi è zero.
2. Accedere al bucket b dove il costo è uguale a 1 essendo che facciamo un accesso alla pagina
3. Accedere alle n tuple tramite puntatori dove il costo è uguale $m \leq n$ dove m sono il numero di pagine.

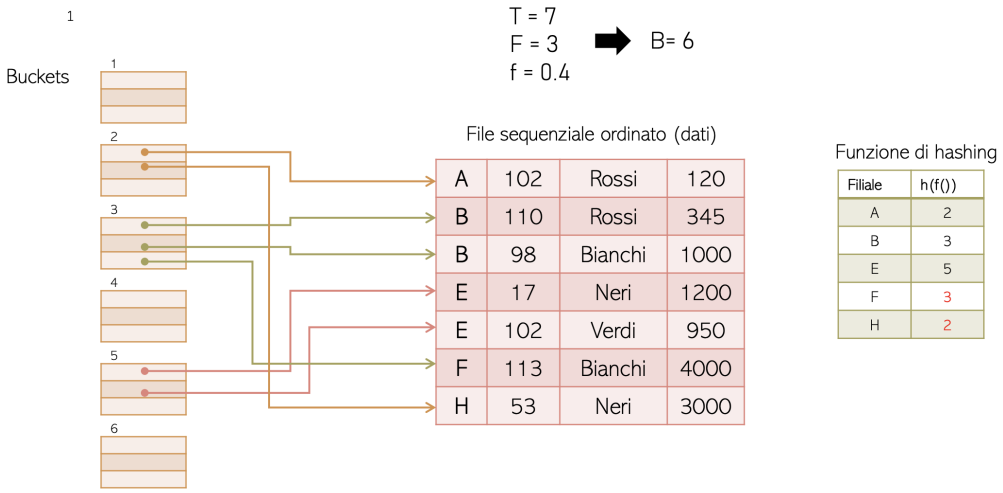


Figure 10: Ricerca di una chiave K in un indice hash

2.4.7 Collisioni

La struttura ad accesso calcolato funziona se i buckets conservano un basso coefficiente di riempimento. Infatti il problema delle strutture ad accesso calcolato è la gestione delle collisioni.

Definizione 2.2: Collisione

Si verifica quando, dati due valori di chiave K_1 e K_2 con $K_1 \neq K_2$, risulta:

$$h(f(K_1)) = h(f(K_2))$$

La probabilità che uno stesso bucket riceva t chiavi su n inserimenti è:

$$p(t) = \binom{n}{t} \left(\frac{1}{B}\right)^t \left(1 - \frac{1}{B}\right)^{n-t}$$

dove B è il numero totale di bucket. La probabilità di avere più di F collisioni, con F il fattore blocco, cioè il numero di puntatori nel bucket, è:

$$p_K = 1 - \sum_{i=0}^{F} p(i)$$

Gestione delle collisioni

Un numero eccessivo di collisioni porta alla saturazione del bucket corrispondente. Per gestirlo si introducono dei **bucket di overflow** collegati al bucket originale, che contengono i puntatori alle tuple che hanno causato la saturazione del bucket originale. Le prestazioni della ricerca peggiorano perchè bisogna accedere ai bucket di overflow.

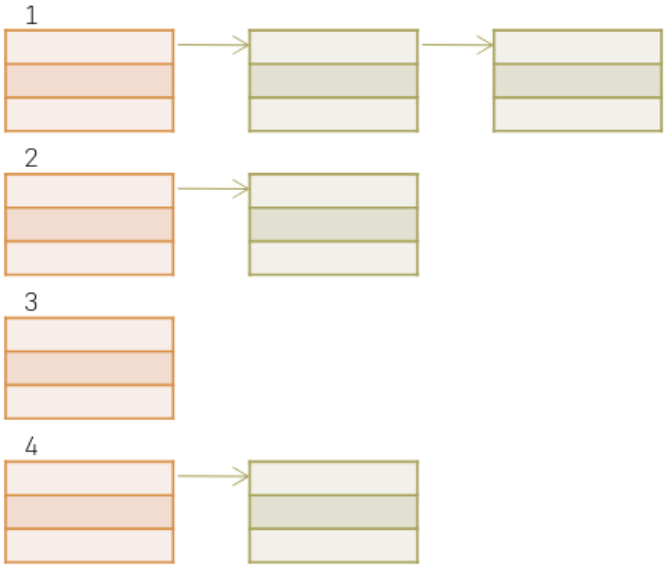


Figure 11: Esempio di bucket di overflow per gestire le collisioni in una struttura ad accesso calcolato

3 Transazioni concorrenti

Il componente dell'architettura del DBMS che si occupa di gestire le transazioni concorrenti è il **gestore della concorrenza** che riceve richieste di accesso ai dati e decide se autorizzarle o meno, eventualmente riordinandol o ritardandole, come se fosse uno scheduler.

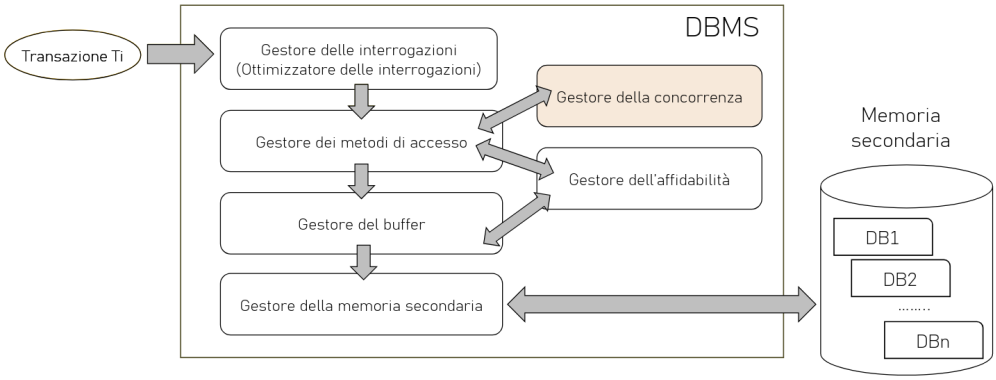


Figure 12: Gestore della concorrenza nell'architettura di un DBMS

L'unità di misura per caratterizzare il carico applicativo di un DBMS è il numero di transazioni al secondo (TPS, Transactions Per Second). Per gestire un carico elevato, tipico delle applicazioni gestionali (100 o 1000 tps), un DBMS deve eseguire le transazioni in modo concorrente.

3.1 Anomalie

L'esecuzione concorrente di transazioni senza controllo può generare **anomalie** o problemi di correttezza ed è quindi necessario introdurre dei meccanismi di controllo nell'esecuzione delle transazioni per evitare queste anomalie.

Definizione 3.1: Notazione

Verrà utilizzata la seguente notazione:

- t_i indica una transazione
- $r_i(x)$ è un operazione di lettura eseguita dalla transazione t_i sulla risorsa x
- $w_i(x)$ è un operazione di scrittura eseguita dalla transazione t_i sulla risorsa x
- `bot` indica l'inizio di una transazione (begin of transaction)
- `commit` indica l'applicazione definitiva degli effetti di una transazione (commit)
- `eot` indica la fine di una transazione (end of transaction)

v

3.1.1 Perdita di aggiornamento

Questa anomalia si verifica quando gli effetti di una transazione concorrente sono persi. Si considerino le seguenti transazioni:

$$t_1 : bot;\ r_1(x);\ x = x + 1;\ w_1(x);\ commit;\ eot$$
$$t_2 : bot;\ r_2(x);\ x = x + 1;\ w_2(x);\ commit;\ eot$$

L'esecuzione concorrente di t_1 e t_2 può portare alla perdita di aggiornamento, come si può vedere dalla seguente simulazione:

Operazioni eseguite da t1	Segmento di memoria centrale di t1	Memoria secondaria [buffer]	Segmento di memoria centrale di t2	Operazioni eseguite da t2
Stato iniziale		x = 2	Stato iniziale	
bot		2		
$r_1(x)$	2 ←	[2]		
x=x+1	3	[2]		bot
	3	[2]	→ 2	$r_2(x)$
	3	[2]	3	x=x+1
	3	[3]	← 3	$w_2(x)$
	3		3	commit
	3			eot
$w_1(x)$	3 →	[3]		
commit	3	3		
eot		3		

Il valore finale di x è 3, mentre una qualsiasi esecuzione seriale avrebbe prodotto per x il valore 4!

Figure 13: Esempio di perdita di aggiornamento

3.1.2 Lettura inconsistente

Questa anomalia si verifica quando accessi successivi ad uno stesso dato, all'interno di una transazione, ritornano valori diversi. Si considerino le seguenti transazioni:

$$t_1 : bot;\ r_1(x);\ r_1'(x);\ commit;\ eot$$
$$t_2 : bot;\ r_2(x);\ x = x + 1;\ w_2(x);\ commit;\ eot$$

L'esecuzione concorrente di t_1 e t_2 può portare alla lettura inconsistente, come si può vedere dalla seguente simulazione:

Operazioni eseguite da t1	Segmento di memoria centrale di t1	Memoria secondaria [buffer]	Segmento di memoria centrale di t2	Operazioni eseguite da t2
Stato iniziale		x = 2	Stato iniziale	
bot		2		
$r_1(x)$	2 ←	[2]		
	2	[2]		bot
	2	[2] →	2	$r_2(x)$
	2	[2]	3	x=x+1
	2	[3]	3	$w_2(x)$
	2	3	3	commit
	2	3		eot
$r_1'(x)$	3 →	[3]		
commit	3	3		
eot		3		

Il valore di x è diventato 3, ma la transazione t_1 non ha modificato x!

Figure 14: Esempio di lettura inconsistente

3.1.3 Lettura sporca

Questa anomalia si verifica quando viene letto un dato che rappresenta uno stato intermedio nell'evoluzione di una transazione. Si considerino le seguenti transazioni:

$$t_1 : bot;\ r_1(x);\ commit;\ eot$$
$$t_2 : bot;\ r_2(x);\ x = x + 1;\ w_2(x);\ \ldots abort;$$

L'esecuzione concorrente di t_1 e t_2 può portare alla lettura sporca, come si può vedere dalla seguente simulazione:

Operazioni eseguite da T1	Segmento di memoria centrale di T1	Memoria secondaria [buffer]	Segmento di memoria centrale di T2	Operazioni eseguite da T2
Stato iniziale		x = 2	Stato iniziale	
bot		2		
		2		bot
		[2] →	2	$r_2(x)$
		[2]	3	x=x+1
		[3] ←	3	$w_2(x)$
$r_1(x)$	3 ←	[3]	3	abort
commit	3	2		
eot		2		

Il valore di x letto da t_1 è 3, ma la transazione t_2 non ha ancora eseguito il commit!

Figure 15: Esempio di lettura sporca

3.1.4 Aggiornamento fantasma

Questa anomalia si verifica quando avviene l'osservazione di uno stato intermedio che non soddisfa i vincoli di integrità. Si considerino le seguenti transazioni:

$$Risorse : x, y, z$$
$$Vincoli : x + y + z = 100$$

$$t_1 : bot;\ r_1(y);\ r_1(x);\ r_1(z);\ s = x + y + z;\ eot$$
$$t_2 : bot;\ r_2(y);\ y = y + 10;\ r_2(z);\ z = z - 10;\ w_2(y);\ w_2(z);\ commit;\ eot$$

L'esecuzione concorrente di t_1 e t_2 può portare all'aggiornamento fantasma, come si può vedere dalla seguente simulazione:

Operazioni eseguite da T1	Segmento di memoria centrale di T1	Memoria secondaria [buffer]	Segmento di memoria centrale di T2	Operazioni eseguite da T2
Stato iniziale		(x,y,z)=20,20,60	Stato iniziale	
bot		20,20,60		
r1 (y)	-,20,-	20,[20],60		
	-,20,-	20,[20],60		bot
	-,20,-	20,[20],60	-,20,-	r2 (y)
	-,20,-	20,[20],60	-,30,-	y = y + 10
	-,20,-	20,[20],[60]	-,30,60	r2 (z)
	-,20,-	20,[20],[60]	-,30,50	z = z - 10
	-,20,-	20,[30],[60]	-,30,50	w2 (y)
	-,20,-	20,[30],[50]	-,30,50	w2 (z)
	-,20,-	20,30,50	-,30,50	commit
	-,20,-	20,30,50		eot
r1 (x)	20,20,-	[20],30,50		
r1 (z)	20,20,50	[20],30,[50]		
s=x-y-z	20,20,50	[20],30,[50]		
eot		20,30,50		

S = 90 → VIOLAZIONE VINCOLO

Figure 16: Esempio di aggiornamento fantasma

3.1.5 Inserimento fantasma

Questa anomalia si verifica quando avvengono valutazioni diverse di valori aggregati a causa di inserimenti intermedi. La transazione che valuta un valore aggregato relativo a tutti gli elementi che soddisfano un predicato di selezione (ad esempio il voto medio degli studenti del terzo anno) può subire l'anomalia di inserimento fantasma se il valore aggregato viene valutato due volte e tra la prima e la seconda valutazione viene inserito un nuovo studente, quindi i due valori medi letti potrebbero essere diversi. Questa anomalia non si può risolvere facendo riferimento solo ai dati già presenti nella base di dati, ma è necessario tener presente che potrebbero esserci delle nuove tuple che compaiono *improvvisamente*.

3.2 Schedule

Uno schedule è una sequenza di operazioni di lettura e scrittura eseguite su risorse della base di dati da diverse transazioni concorrenti che rappresenta una possibile esecuzione concorrente di diverse transazioni:

$$S : r_1(x);\ r_2(z);\ w_1(x);\ w_2(z);$$

Per evitare anomalie si stabilisce di accettare **solo** gli schedule equivalenti ad uno schedule seriale.

3.2.1 Schedule seriale

Lo schedule seriale è uno schedule dove le operazioni di ogni transazione compaiono in sequenza **senza essere inframezzate** da operazioni di altre transazioni.

Esempio 3.1

Consideriamo i seguenti esempi di schedule:

- $S_1 : r_1(x);\ r_2(x);\ w_2(x);\ w_1(x);$ **non** è seriale
- $S_2 : r_1(x);\ w_1(x);\ r_2(x);\ w_2(x);$ **è** seriale

Definizione 3.2: Schedule serializzabile

Lo schedule serializzabile è uno schedule equivalente ad uno schedule seriale. Due schedule sono equivalenti se **producono gli stessi effetti** sulla base di dati.

3.3 Tecniche di gestione della concorrenza

Per le seguenti tecniche di gestione della concorrenza, si assume l'ipotesi di **commit-proiezione**, cioè si suppone che le transazioni abbiano esito noto. Quindi si tolgono dagli schedule tutte le operazioni delle transazioni che non vanno a buon fine.

3.3.1 View-equivalenza

Definizione 3.3: Relazione LEGGE_DA

Dato uno schedule S si dice che un'operazione di lettura $r_i(x)$, che compare in S, `LEGGE_DA` un'operazione di scrittura $w_j(x)$, che compare in S, se $w_j(x)$ **precede** $r_i(x)$ in S e non vi è alcuna operazione di scrittura $w_k(x)$ tra le due. Con $j \neq i$.

Esempio 3.2

Un esempio di utilizzo della relazione LEGGE_DA è il seguente:

- $S_1 : r_1(x);\ r_2(x);\ w_2(x);\ w_1(x);\ \texttt{LEGGE_DA}(S_1) = \varnothing$
- $S_2 : r_1(x);\ w_1(x);\ r_2(x);\ w_2(x);\ \texttt{LEGGE_DA}(S_2) = \{(r_2(x), w_1(x))\}$
- $S_3 : r_1(x);\ w_1(x);\ w_2(x);\ r_2(x);\ \texttt{LEGGE_DA}(S_3) = \varnothing$

Definizione 3.4: Scritture finali

Dato uno schedule S si dice che un'operazione di scrittura $w_i(x)$, che compare in S, è una **SCRITTURA FINALE** se è l'ultima operazione di scrittura della risorsa x in S.

Esempio 3.3

Un esempio di utilizzo della relazione SCRITTURE_FINALI è il seguente:

- $S_1 : r_1(x);\ r_2(x);\ w_2(x);\ w_3(y);\ w_1(x);$
  $\texttt{SCRITTURE_FINALI}(S_1) = \{w_3(y), w_1(x)\}$
- $S_2 : r_1(x);\ w_1(x);\ r_2(x);\ w_2(x);\ w_3(y);$
  $\texttt{SCRITTURE_FINALI}(S_2) = \{w_2(x), w_3(y)\}$

Definizione 3.5: View-equivalenza

Due schedule S_1 e S_2 sono view-equivalenti ($S_1 \approx_V S_2$) se possiedono le stesse relazioni LEGGE_DA e SCRITTURE_FINALI.

Definizione 3.6: View-serializzabilità

Uno schedule S è view-serializzabile (VSR) se esiste uno schedule seriale S' tale che $S \approx_V S'$.

Attenzione: gli schedule seriali S' si generano considerando tutte le possibili permutazioni **delle transazioni** (**non** delle operazioni delle transazioni) che compaiono in S. Inoltre **non si deve** cambiare l'ordine delle operazioni di ciascuna transazione perchè ciò significherebbe modificare la transazione.

Gli schedule corrispondenti alle anomalie di:

- Perdita di aggiornamento
- Letture inconsistenti
- Aggiornamenti fantasma

non sono view-serializzabili. Inoltre l'algoritmo per il test di view-equivalenza tra due schedule è di complessità lineare, mentre l'algoritmo per il test di view-serializzabilità di uno schedule è di complessità esponenziale.

In conclusione si è osservato che la view-serializzabilità richiede l'ipotesi di commit-proiezione, e algoritmi di complessità troppo elevata, quindi non è applicabile nei sistemi reali.

Esempio 3.4

Consideriamo le seguenti transazioni:

$$t_1 : r_1(x);\ w_1(x)$$
$$t_2 : r_2(x);\ w_2(x)$$

Eseguite con lo schedule:

$$S_{pa} = r_1(x);\ r_2(x);\ w_2(x);\ w_1(x);$$

Questo schedule è view serializzabile?

Calcoliamo le relazioni:

- $\texttt{LEGGE_DA}(S_{pa}) = \varnothing$
- $\texttt{SCRITTURE_FINALI}(S_{pa}) = \{w_1(x)\}$

Bisogna generare tutti i possibili schedule seriali per determinare se esiste uno schedule seriale S' tale che $S_{pa} \approx_V S'$. Gli schedule seriali sono:

- $S_{12} = r_1(x);\ w_1(x);\ r_2(x);\ w_2(x);$
  - $\texttt{LEGGE_DA}(S_{12}) = \{(r_2(x), w_1(x))\}$
  - $\texttt{SCRITTURE_FINALI}(S_{12}) = \{w_1(x)\}$

  Osserviamo che $S_{pa} \not\approx_V S_{12}$
- $S_{21} = r_2(x);\ w_2(x);\ r_1(x);\ w_1(x);$
  - $\texttt{LEGGE_DA}(S_{21}) = \{(r_1(x), w_2(x))\}$
  - $\texttt{SCRITTURE_FINALI}(S_{21}) = \{w_1(x)\}$

  Osserviamo che $S_{pa} \not\approx_V S_{21}$

Avendo provato per tutte le possibili permutazioni si può dire che lo schedule S_{pa} **non è view-serializzabile**.

Esempio 3.5 (Esercizio di lettura inconsistente)

onsideriamo le seguenti transazioni:

$$t_1 : r_1(x);\ r'_1(x)$$
$$t_2 : r_2(x);\ w_2(x)$$

Eseguite con lo schedule:

$$S_{li} = r_1(x);\ r_2(x);\ w_2(x);\ r'_1(x);$$

Questo schedule è view serializzabile?

Calcoliamo le relazioni:

- $\texttt{LEGGE_DA}(S_{li}) = \{(r'_1(x), w_2(x))\}$
- $\texttt{SCRITTURE_FINALI}(S_{li}) = \{w_2(x)\}$

Bisogna generare tutti i possibili schedule seriali per determinare se esiste uno schedule seriale S' tale che $S_{li} \approx_V S'$. Gli schedule seriali sono:

- $S_{12} = r_1(x);\ r'_1(x);\ r_2(x);\ w_2(x);$
  - $\texttt{LEGGE_DA}(S_{12}) = \varnothing$
  - $\texttt{SCRITTURE_FINALI}(S_{12}) = \{w_2(x)\}$

  Osserviamo che $S_{li} \not\approx_V S_{12}$
- $S_{21} = r_2(x);\ w_2(x);\ r_1(x);\ r'_1(x);$
  - $\texttt{LEGGE_DA}(S_{21}) = \{(r_1(x), w_2(x)), (r'_1(x), w_2(x))\}$
  - $\texttt{SCRITTURE_FINALI}(S_{21}) = \{w_2(x)\}$

  Osserviamo che $S_{li} \not\approx_V S_{21}$

Avendo provato per tutte le possibili permutazioni si può dire che lo schedule S_{li} **non è view-serializzabile**.

Esempio 3.6

onsideriamo le seguenti transazioni:

$$t_1 : r_1(x);\ w_1(x);\ r_1(y);\ w_1(y)$$
$$t_2 : r_2(z);\ r_2(x);\ w_2(x);\ w_2(z)$$

Eseguite con lo schedule:

$$S_1 = r_1(x);\ w_1(x);\ r_2(z);\ r_1(y);\ w_1(y);\ r_2(x);\ w_2(x);\ w_2(z);$$

Questo schedule è view serializzabile?

Calcoliamo le relazioni:

- `LEGGE_DA`$(S_1) = \{(r_2(x), w_1(x))\}$
- `SCRITTURE_FINALI`$(S_1) = \{w_2(x), w_1(y), w_2(z)\}$

Bisogna generare tutti i possibili schedule seriali per determinare se esiste uno schedule seriale S' tale che $S_1 \approx_V S'$. Gli schedule seriali sono:

- $S_{12} = t_1 t_2$
- $S_{21} = t_2 t_1$

Al posto di creare tutte le permutazioni degli schedule seriali, cerchiamo di osservare le relazioni LEGGE_DA e SCRITTURE_FINALI per capire se S_1 è view-equivalente ad S_{12} o S_{21}.
Dalla presenza di $w_2(x)$ nelle scritture finali di S_1 si nota che $t_1 < t_2$ perchè $w_2(x)$ è l'ultima scrittura di x in S_1, e questa operazione viene eseguita solo da t_2 e di conseguenza va eseguita dopo t_1. Da ciò sappiamo che sicuramente $S_1 \not\approx_V S_{21}$ e ci rimane da controllare soltanto se $S_1 \approx_V S_{12}$:

$$S_{12} = r_1(x);\ w_1(x);\ r_1(y);\ w_1(y);\ r_2(z);\ r_2(x);\ w_2(x);\ w_2(z);$$

- `LEGGE_DA`$(S_{12}) = \{(r_2(x), w_1(x))\}$
- `SCRITTURE_FINALI`$(S_{12}) = \{w_2(x), w_1(y), w_2(z)\}$

Osserviamo che $S_1 \approx_V S_{12}$ e quindi S_1 è view-serializzabile.

Esempio 3.7

Consideriamo le seguenti transazioni:

$$t_1 : r_1(x);\ w_1(x);\ w_1(y)$$
$$t_2 : r_2(y);\ r_2(x)$$
$$t_3 : w_3(x);\ r_3(y);\ w_3(y)$$

$$S_2 = r_1(x);\ w_1(x);\ w_3(x);\ r_2(y);\ r_3(y);\ w_3(y);\ w_1(y);\ r_2(x)$$

Questo schedule è view serializzabile?

Calcoliamo le relazioni:

- $\texttt{LEGGE_DA}(S_2) = \{(r_2(x), w_3(x))\}$
- $\texttt{SCRITTURE_FINALI}(S_2) = \{w_3(x), w_1(y)\}$

Siccome ci sono 6 possibili permutazioni vediamo se è possibile scartarne alcune a priori.

- L'insieme $\texttt{SCRITTURE_FINALI}(S_2)$ implica:
  - $t_1 < t_3$ perchè $w_3(x)$ è l'ultima scrittura di x in S_2 e questa operazione viene eseguita solo da t_3 e di conseguenza va eseguita dopo t_1
  - $t_3 < t_1$ perchè $w_1(y)$ è l'ultima scrittura di y in S_2 e questa operazione viene eseguita solo da t_1 e di conseguenza va eseguita dopo t_3

  Si osserva che $t_1 < t_3$ e $t_3 < t_1$ è una contraddizione, cioè non esiste uno schedule seriale che le rispetti entrambe e quindi si può concludere che S_2 **non è view-serializzabile**.

- L'insieme $\texttt{LEGGE_DA}(S_2)$ implica:
  - $t_3 < t_2$ perchè $r_2(x)$ (in t_2) legge da $w_3(x)$ (in t_3) e quindi $(r_2(x), w_3(x)) \in \texttt{LEGGE_DA}$
  - $t_1 < t_3$ per garantire $(r_1(x), w_3(x)) \notin \texttt{LEGGE_DA}$
  - $t_2 < t_3$ per garantire $(r_2(x), w_3(x)) \in \texttt{LEGGE_DA}$
  - $t_2 < t_1$ per garantire $(r_2(x), w_1(x)) \notin \texttt{LEGGE_DA}$
  - $t_3 < t_1$ per garantire $(r_3(x), w_1(x)) \notin \texttt{LEGGE_DA}$

  Si osserva che $t_1 < t_3$ e $t_3 < t_1$ è una contraddizione, quindi si può concludere che S_2 **non è view-serializzabile**.

Esempio 3.8

Consideriamo le seguenti transazioni:

$$
\begin{aligned}
&t_1 : r_1(x);\ w_1(x);\ w_1(y);\ w_1(z)\\
&t_2 : r_2(x);\ w_2(x)\\
&t_3 : r_3(x);\ w_3(y);\ w_3(x)\\
&t_4 : r_4(z)\\
&t_5 : w_5(x);\ w_5(y);\ r_5(z)
\end{aligned}
$$

eseguite con lo schedule:

$$
\begin{aligned}
S_3 = &r_1(x);\ r_2(x);\ w_2(x);\ r_3(x);\ r_4(z);\ w_1(x);\ w_3(y);\ w_3(x);\ w_1(y);\\
&w_5(x);\ w_1(z);\ w_5(y);\ r_5(z)
\end{aligned}
$$

Questo schedule è view serializzabile?

Calcoliamo le relazioni:

- `LEGGE_DA`$(S_3) = \{(r_3(x), w_2(x)), (r_5(z), w_1(z))\}$
- `SCRITTURE_FINALI`$(S_3) = \{w_5(x), w_5(y), w_1(z)\}$

Le permutazioni di tutte le transazioni sono 5! e quindi è necessario cercare di scartarne alcune:

- Considerando `SCRITTURE_FINALI`(S_3) si ha che:
  - $w_5(x) \Rightarrow t_2 < t_5,\ t_1 < t_5,\ t_3 < t_5$
  - $w_5(y) \Rightarrow t_3 < t_5,\ t_1 < t_5$

  Da queste informazioni non si rilevano contraddizioni, quindi si procede con l'analisi di `LEGGE_DA`(S_3).
- Considerando `LEGGE_DA`(S_3) si ha che:
  - $(r_3(x), w_2(x)) \Rightarrow t_2 < t_3$
  - $(r_5(z), w_1(z)) \Rightarrow t_1 < t_5$

  Da queste informazioni non si rilevano contraddizioni, quindi si procede osservando cosa **non compare** nell'insieme `LEGGE_DA`(S_3):
  - $(r_1(x), w_2(x)) \notin$ `LEGGE_DA`$(S_3) \Rightarrow t_1 < t_2$
  - $(r_1(x), w_3(x)) \notin$ `LEGGE_DA`$(S_3) \Rightarrow t_1 < t_3$

- $(r_1(x), w_5(x)) \notin \texttt{LEGGE_DA}(S_3) \Rightarrow t_1 < t_5$
- $(r_2(x), w_1(x)) \notin \texttt{LEGGE_DA}(S_3) \Rightarrow t_2 < t_1$ Questo è in contraddizione con $t_1 < t_2$ e si può concludere che S_3 **non è view-serializzabile**.

3.3.2 Conflict-equivalenza

Definizione 3.7: Conflitto

Dato uno schedule S si dice che una coppia di operazioni (a_i, a_j), dove a_i e a_j compaiono nello schedule S, rappresentano un conflitto se:

- appartengono a transazioni diverse $(i \neq j)$
- entrambe operano sulla stessa risorsa
- almeno una di esse è un'operazione di scrittura
- a_i compare in S prima di a_j

Definizione 3.8: Conflict-equivalenza

Due schedule S_1 e S_2 sono conflict-equivalenti $(S_1 \approx_C S_2)$ se possiedono le stesse operazioni e gli stessi conflitti, cioè se le operazioni in conflitto sono nello stesso ordine nei due schedule.

Definizione 3.9: Conflict-serializzabilità

Uno schedule S è conflict-serializzabile (CSR) se esiste uno schedule seriale S' tale che $S \approx_C S'$.

L'algoritmo per il test di conflict-serializzabilità di uno schedule è di **complessità lineare** ed è il seguente:

Dato uno schedule S:

- Si costruisce il grafo dei conflitti $G(N, A)$ dove:
  - $N = \{t_1, \ldots, t_n\}$ è l'insieme delle transazioni di S
  - $(t_i, t_j) \in A$ se esiste almeno un conflitto (a_i, a_j) in S
- Se il grafo è **aciclico**, allora S è conflict-serializzabile.

Richiede comunque l'ipotesi di commit-proiezione.

Teorema 3.3.1

Se S è CSR (ovvero $\exists S_r$ seriale tale che $S \approx_C S_r$) allora il suo grafo è **aciclico**.

Proof. Ipotesi: le transazioni sono ordinate $t_1, \ldots, t_n$ e poiché $S \approx_C S_r$, S_r ha tutti i conflitti di S con operaizoni esattamente nello stesso ordine. Nel grafo dei conflitti di S_r ci possono essere solo archi i, j con $i < j$, quindi non possono esserci cicli nel grafo di S_R perché un ciclo richiederebbe un arco j, i con $j > i$ che torna indietro. Siccome $S \approx_C S_R$ allora hanno lo stesso grafo dei conflitti, quindi anche il grafo dei conflitti di S è aciclico.

Teorema 3.3.2

Se il grafo S è aciclico allora S è CSR.

Proof. Ipotesi: il grafo dei conflitti di S è aciclico. Vuol dire che esiste un ordinamento topologico cioè una numerazione dei nodi tale che il grafo contiene solo archi (i, j) con $i < j$. Utilizzando questo ordinamento topologico possiamo definire delle transazioni che produce lo stesso grafo allora vuol dire che abbiamo uno schedule seriale.

Esempio 3.9

Consideriamo le seguenti transazioni:

$$t_1 : r_1(x)\ w_1(x) \quad t_2 : r_2(x)\ w_2(x)$$

E lo schedule:

$$S_{pa} : r_1(x)\ r_2(x)\ w_2(x)\ w_1(x)$$

I conflitti sono i seguenti:

$$\text{conflitti}(S_{pa}) = \{(r_1(x), w_2(x)), (r_2(x), w_1(x)), (w_2(x), w_1(x))\}$$

dove:

- $(r_1(x), w_2(x))$ è un conflitto perché $r_1(x)$ e $w_2(x)$ operano sulla stessa risorsa x,
- $(r_2(x), w_1(x))$ è un conflitto perché $r_2(x)$ e $w_1(x)$ operano sulla stessa risorsa x.
- $(w_2(x), w_1(x))$ è un conflitto perché $w_2(x)$ e $w_1(x)$ operano sulla stessa risorsa x.

Il grafo dei conflitti è il seguente:

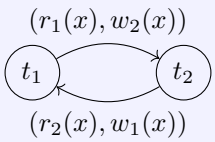


Figure 17: Grafo dei conflitti di S_{pa}

Il grafo dei conflitti di S_{pa} è ciclico, quindi S_{pa} **non è conflict-serializzabile**.

Teorema 3.3.3

La conflict-serializzabilità è una condizione sufficiente, ma non necessaria per la view-serializzabilità, cioè $CSR \subset VSR$:

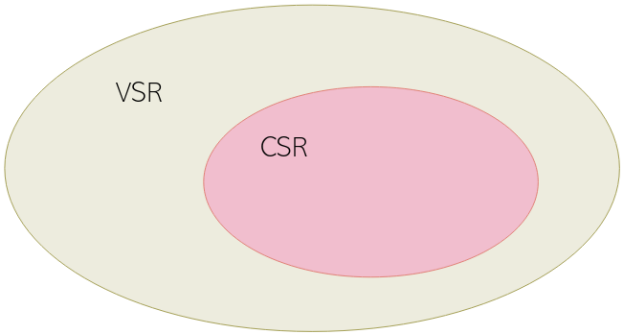


Figure 18: Relazione tra conflict-serializzabilità e view-serializzabilità

Proof. Per $CSR \subset VSR$:

- Basta trovare $S \in VSR$ ma $S \notin CSR$
- $$S = r_1(x);\ w_2(x);\ w_1(x);\ w_3(x);$$
  - `LEGGE_DA` $= \varnothing$
  - `SCRITTURE_FINALI` $= \{w_3(x)\}$

  Questo è VSR perchè $\exists S_R . S_r \approx_V S$

  $$S_R = r_1(x);\ w_1(x);\ w_2(x);\ w_3(x);$$

- LEGGE_DA$(S_r) = \varnothing$
- SCRITTURE_FINALI$(S_r) = \{w_3(x)\}$

- Dove i conflitti sono:

$$\text{Conflitti}(S_{Pa}) = \begin{aligned} &(r_1(x), w_2(x)), (r_1(x), w_3(x)), \\ &(w_2(x), w_1(x)), (w_2(x), w_3(x)), \\ &(w_1(x), w_3(x)) \end{aligned}$$

  Quindi il grafo:

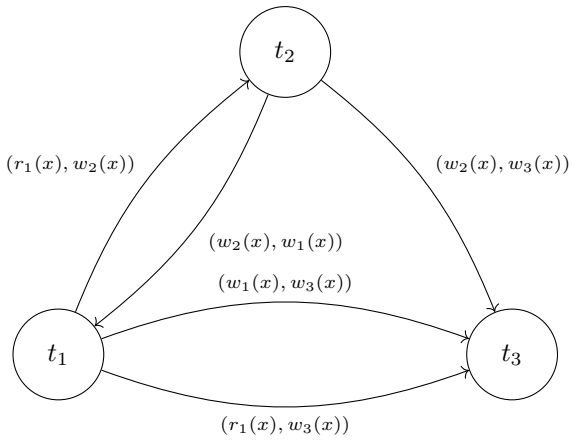


Figure 19: Grafo dei conflitti di S_{Pa}

Per $VSR \not\subset CSR$: Dato $S_1 \approx S_2$ e dimostriamo $S_1 \approx_V S_2$:

- Devono avere lo stesso insieme di SCRITTURE_FINALI, altrimenti ci sarebbero due scritture sulla stessa risorsa da parte di transazioni diverse in un ordine diverso e quindi un conflitto diverso.
- Devono avere lo stesso insieme di LEGGE_DA, altrimenti ci sarebbe un conflitto composto da una lettura ed una scrittura diversa e quindi un conflitto diverso.
- Se S:
  - è CSR allora S è VSR
  - non è CSR allora S non è VSR

Esempio 3.10

Consideriamo il seguente schedule:

$$S = r_(x)\ w_1(x)\ r_2(z)\ r_1(y)\ w_1(y)\ r_2(x)\ w_2(x)\ w_2(z)$$

Ci chiediamo se S è CSR e di conseguenza se è VSR.

$$\begin{aligned} & (r_1(x), w_2(x)), \\ \text{conflitti}(S) = & (w_1(x), r_2(x)), \\ & (w_1(y), w_2(y)) \end{aligned}$$

dove il grafo dei conflitti è il seguente:

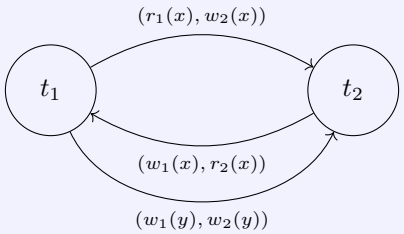


Figure 20: Grafo dei conflitti di S

Il grafo è acilico e quindi S_1 è CSR e di conseguenza è anche VSR.

Esempio 3.11

$$S_2 = r_(x)\ w_1(x)\ w_3(x)\ r_2(y)\ r_3(y)\ w_3(y)\ w_1(y)\ r_2(x)$$

I conflitti sono:

$$\begin{aligned} & (r_1(x), w_3(x)), \\ & (w_1(x), w_3(x)), (w_1(x), r_2(x)) \\ \text{conflitti}(S) = & (w_3(x), r_2(x)) \\ & (r_2(y), w_3(y)), (r_2(y), w_1(y)) \\ & (r_3(y), w_1(y)), (w_3(y), w_1(y)) \end{aligned}$$

Il grafo dei conflitti è il seguente:

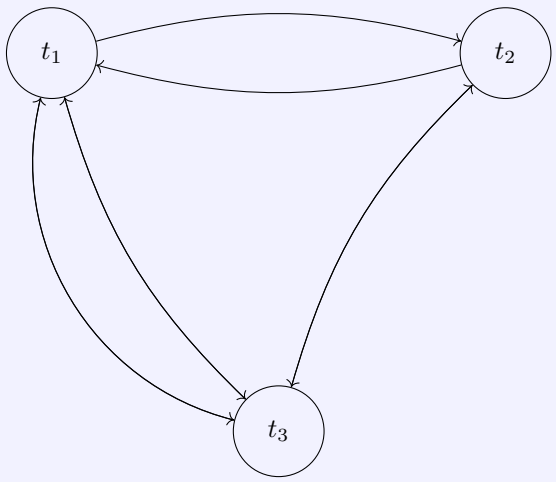


Figure 21: Grafo dei conflitti di S_2

Il grafo è ciclico e qundi S_2 non è CSR e di conseguenza non è VSR.

Esempio 3.12

Consideriamo la schedule:

$$S_3 = r_1(x)\ r_2(x)\ w_2(x)\ r_3(x)\ r_4(z)\ w_1(x)\ w_3(y)\ w_3(x)\ w_1(y)\ w_5(x)\ w_1(z)\ w_5(y)\ r_5(z)$$

I conflitti sono:

$$\begin{aligned}
& \{(r_1(x), w_2(x)), \\
& (r_1(x), w_3(x)), (r_1(x), w_5(x)) \\
& (r_2(x), w_1(x)), (r_2(x), w_3(x)), (r_2(x), w_5(x)) \\
& (w_2(x), r_3(x)), (w_2(x), w_1(x)), (w_2(x), w_3(x)), (w_2(x), w_5(x)) \\
\text{conflitti}(S_3) = & (r_3(x), w_1(x)), (r_3(x), w_5(x)) \\
& (r_4(z), w_1(z)), \\
& (w_1(x), w_3(x)), (w_1(x), w_5(x)) \\
& (w_3(y), w_1(y)), (w_3(y), w_5(y)) \\
& (w_3(x), w_5(x))\}
\end{aligned}$$

Il grafo dei conflitti è il seguente:

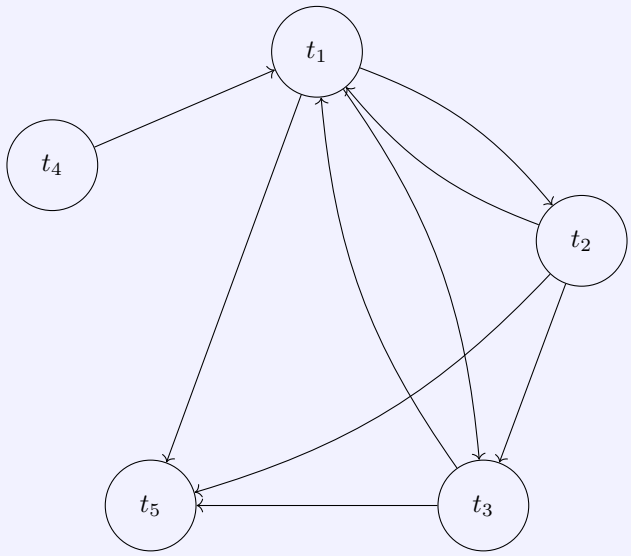


Figure 22: Grafo dei conflitti di S_3

Il grafo è ciclico e quindi S_3 non è CSR e di conseguenza non è VSR.

3.3.3 Locking a due fasi stretto

Il locking a due fasi (2PL, Two-Phase Locking) è il metodo applicato nei sistemi reali per la gestione dell'esecuzione concorrente delle transazioni perchè non richiede di conoscere in anticipo l'esito delle transazioni. Il locking a due fasi è un sottoinsieme della conflict-serializzabilità. Gli aspetti che caratterizzano il locking a due fasi sono i seguenti:

- **Meccanismo di base per la gestione dei lock**: Si basa sull'introduzione di alcune **primitive di lock** che consentono alle transazioni di bloccare le risorse sulle quali vogliono agire con operazioni di lettura e scrittura. Le primitive di lock sono le seguenti:
  - $r_lock_K(x)$: richiesta di un lock **condiviso** da parte della transazione t_K sulla risorsa x per eseguire una lettura
  - $w_lock_K(x)$: richiesta di un lock **esclusivo** da parte della transazione t_K sulla risorsa x per eseguire una scrittura
  - $unlock_K(x)$: richiesta da parte della transazione t_K di liberare la risorsa x da un precedente lock

  Le transazioni devono seguire le seguenti regole per applicare le primitive di lock:

  - Ogni **lettura** deve essere preceduta da un *r_lock* e seguita da un *unlock*. Sono ammessi più *r_lock* contemporanei sulla stessa risorsa (lock condiviso)

– Ogni **scrittura** deve essere preceduta da un *w_lock* e seguita da un *unlock*. **Non** sono ammessi più *w_lock* (oppure *w_lock* e *r_lock*) contemporanei sulla stessa risorsa (lock esclusivo)

Se una transazione segue le precedenti regole, allora si dice **ben formata** rispetto al locking.

- **Politica di concessione dei lock sulle risorse**: Il gestore dei lock (o gestore della concorrenza) mantiene per ogni risorsa x le seguenti informazioni:

  – **Stato**: $s(x) \in \{libero, r_lock, w_lock\}$

  – **Transazioni in r_lock**: $c(x) = (t_1, \ldots, t_n)$, dove $t_1, \ldots, t_n$ hanno un *r_lock* su x

  Il comportamento del gestore dei lock è descritto dalla seguente tablella:

Stato x / Richiesta	LIBERO		R_LOCK		W_LOCK	
$r_lock_K(x)$	Esito OK	Operazioni s(x) = r_lock c(x) ={k}	Esito OK	Operazioni c(x) = c(x)∪{k}	Esito Attesa	Operazioni -
$w_lock_K(x)$	Esito OK	Operazioni s(x) = w_lock	if |c(x)|=1 and k∈c(x) then Esito OK	Operazioni s(x) = w_lock	Esito Attesa	Operazioni -
			else Attesa	-		
$unlock_K(x)$	Esito Errore	Operazioni -	Esito OK	Operazioni c(x) = c(x) – {k} if c(x)=∅ then s(x)=Libero verifica coda	Esito OK	c(x)=∅ s(x)=Libero verifica coda

Table 1: Comportamento del gestore dei lock

- **Regola che garantisce la serializzabilità**: Una transazione **dopo aver rilasciato un lock non può acquisirne altri**. Questa regola è visualizzata dal seguente grafico:

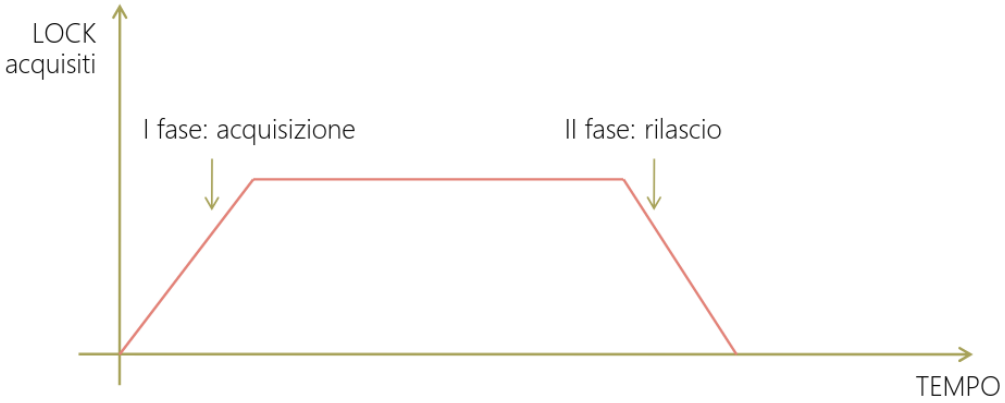


Figure 23: Locking a due fasi

Il locking a due fasi stretto (Strict 2PL) è una variante del locking a due fasi che introduce una condizione aggiuntiva per rimuovere la necessità dell'ipotesi di commit-proiezione. La condizione aggiuntiva è la seguente: Una transazione può rilasciare i lock solo quando ha eseguito correttamente un **commit** o un **rollback**.

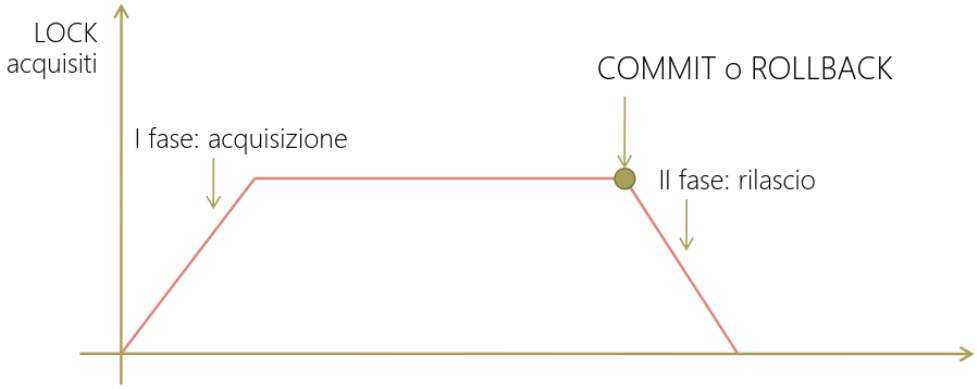


Figure 24: Locking a due fasi stretto

3.3.4 Blocco critico (deadlock)

Il blocco critico è una situazione di blocco che si verifica nell'esecuzione di transazioni concorrenti quando:

- Due transazioni hanno bloccato delle risorse:
  - t_1 ha bloccato r_1
  - t_2 ha bloccato r_2
- Inoltre t_1 è in attesa sulla risorsa r_2 e t_2 è in attesa sulla risorsa r_1

Se il numero medio di tuple per tabella è n e la granularità del lock (cioè il numero di risorse che possono essere bloccate) è la tupla, la probabilità che si verifichi un lock tra due transazioni è:

$$P(\text{deadlock di lunghezza } 2) = \frac{1}{n^2}$$

Esempio 3.13

Un esempio di deadlock utilizzando il locking a due fasi stretto è il seguente:

t1	M.C.	Database			M.C.	t2
		c(x)	s(x)	Val		
bot		∅	L	2		
$r_lock_1(x)$		{1}	RL	2		
$r_1(x)$	2	{1}	RL	2		
x = x +1	3	{1}	RL	2		
	3	{1}	RL	2		bot
	3	{1,2}	RL	2		$r_lock_2(x)$
	3	{1,2}	RL	2	2	$r_2(x)$
	3	{1,2}	RL	2	3	x=x+1
	3	{1,2}	RL	2	3	$w_lock_2(x) \rightarrow$ ATTESA
$w_lock_1(x) \rightarrow$ ATTESA	3	{1,2}	RL	2	3	

BLOCCO CRITICO

Figure 25: Esempio di deadlock con locking a due fasi stretto

Le tecniche per risolvere i deadlock sono le seguenti:

- **Timeout**: se una transazione è in attesa su una risorsa da troppo tempo viene abortita
- **Prevenzione**:
  - Una transazione blocca tutte le risorse a cui deve accedere in lettura o scrittura in una sola volta (o blocca tutto o non blocca nulla)
  - Ogni transazione t_i acquisisce un timestamp TS_i all'inizio della sua esecuzione. La transazione t_i può attendere una risorsa bloccata da t_j solo se vale una certa condizione sui timestamp, ad esempio $TS_i < TS_j$, altrimenti t_i viene abortita e fatta ripartire con lo stesso timestamp
- **Rilevamento**: Si esegue un'analisi periodica della tabella di lock per rilevare la presenza di un deadlock che corrisponde ad un ciclo nel grafo delle relazioni di attesa. Questa tecnica è la più utilizzata nei sistemi reali.

Inoltre la scelta di gestire i lock in lettura come lock condivisi introduce il problema della **starvation** che tuttavia nel contesto delle DMBS risulta poco probabile, ma comunque risolvibile con le tecniche precedenti (in particolare il rilevamento).

Definition 3.1 (Starvation)**.** Se una risorsa x viene costantemente bloccata da una sequenza di transazioni che acquisiscono *r_lock* su x, un'eventuale transazione in attesa di scrivere su x viene bloccata per un lungo periodo fino alla fine della sequenza di letture.

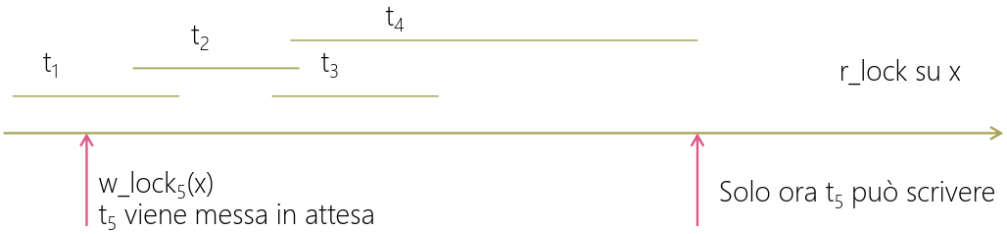


Figure 26: Esempio di starvation

3.3.5 Gestione della concorrenza in SQL

Siccome risulta molto pesante per il sistema gestire l'esecuzione concorrente di transazioni con una conseguente diminuzione delle prestazioni, SQL prevede la possibilità di **rinunciare del tutto o in parte al controllo di concorrenza** per aumentare le prestazioni del sistema. È quindi possibile, a livello di una singola transazione, decidere di **tollerare alcune anomalie** di esecuzione concorrente. La seguente tabella mostra le possibili combinazioni di anomalie che si possono tollerare:

Livello di isolamento	Perdita di update	Lettura sporca	Lettura inconsistente	Update fantasma	Inserimento fantasma
serializable	✓	✓	✓	✓	✓
repeatable read	✓	✓	✓	✓	
read committed	✓	✓			
read uncommitted	✓				

Table 2: Anomalie tollerate in SQL

Nota:

- Tutti i livelli richiedono il 2PL stretto per le scritture
- **Serializable**: Richiede il 2PL stretto anche per le scritture e applica il lock di predicato (cioè blocca tutte le risorse che soddisfano un certo predicato) per evitare l'inserimento fantasma
- **Repeatable read**: Applica il 2PL stretto per tutti i lock in lettura applicati a livello di tupla e non di tabella. Consente inserimenti e quindi non evita l'anomalia di inserimento fantasma

- **Read committed**: Richiede lock condivisi per le letture, ma non il 2PL stretto
- **Read uncommitted**: Non applica lock in lettura